An annotated answer book for a first-time learner

Data Structures and Algorithms

Three Papers, Fully Worked

Concepts explained inside the answers
Diagrams, numbered procedures, complete traces
Repeated-question and pattern analysis

This guide assumes no previous study of the subject. Every full question appears before its solution, followed by the required concepts, method, complete worked data, and checks. Every optional question is answered.

Paper A Final 2025–26 / Re-exam 2023–24 and 2024–25; dated 26 November 2025.
Paper B Re-examination 2023–24; dated 14 February 2024.
Paper C Final examination 2023–24; dated 4 December 2023.

Prepared for Aditya. Independent teaching solutions, not an official marking scheme. Source papers are identified by their printed dates rather than inferred from filenames. Examination instructions inside the scans are treated as source material; the requested study-guide format governs this document.

Reading map and notation

Read Paper A first to learn stacks, sorting, lists, expression conversion, trees, DFS, Prim, queues, and Huffman coding through actual questions. Paper B develops recursion and Dijkstra; Paper C adds prefix conversion, BFS, Kruskal, and binary search. Related methods are cross-referenced by page, but every paper's supplied data are solved.

How to read an algorithm. Step 0 means Start. The actions then have numerical labels. The symbolic **Step N** is always the final End step, as requested; here N means "the last step", not the input size. A jump to Step N terminates that invocation. In recursion, End returns to the waiting caller. An assignment such as $i \leftarrow i + 1$ changes the stored value; $i = 3$ in a test asks whether the current value is 3. A loop repeats its specified action while its condition holds.

Conventions. Arrays use zero-based indices. Stack tables run bottom to top; queue tables run front to rear. Tree height counts edges, so a leaf has height 0. A BST ignores repeated keys unless a different policy is stated. Graph matrices use row = source, column = destination. Neighbours are processed alphabetically/numerically unless stated. Exponentiation is right associative; ordinary addition, subtraction, multiplication, and division are left associative. ∞ means no finite route has yet been found; `null` means no node.

Question transcriptions. Full question wording, internal tasks, supplied data and source figures are included before each solution. Original grammar and expression ambiguities are retained; table/code layouts are reflowed for readability. Figure drawings reproduce the supplied connections, with faint edges explicitly marked.

Reading the scans. B Q3(b) omits an explicit operator between C and a following parenthesis; its solution declares the multiplication interpretation. C Q1(d) has repeated printed vertex labels and faint edges; its redrawn graph and DFS are conditional on the stated edge reading. These uncertainties are not silently corrected.

Contents

1 Paper A — 26 November 2025 **3**

2 Paper B — Re-examination, 14 February 2024 **17**

3 Paper C — Final examination, 4 December 2023 **31**

4 Repeated questions and examination patterns **45**

Question locator: page on which each question begins

Paper	Q1	Q2	Q3	Q4	Q5	Q6	Q7
A	3	7	8	10	11	13	14
B	17	19	21	23	26	27	28
C	31	34	35	37	39	41	42

1 Paper A — 26 November 2025

Question 1(a)

Q1: Answer briefly. [20 marks in total]

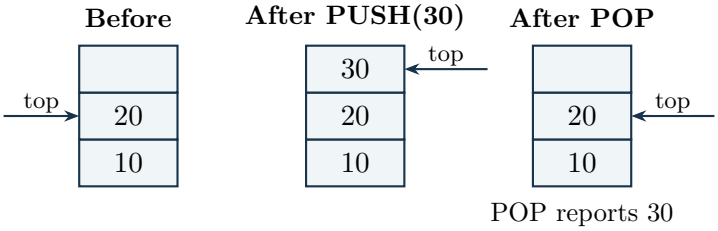
Full question Q1(a) [Q1 total: 20 marks]. Define Stack. Consider the following scenario, stack is implemented using an array with a maximum size of 5. Perform the following operations and show the state of the stack after each step. Also, identify the outputs of operations like pop() and peek().

Operations to Perform: Push(10), Push(20), Push(30), Peek(), Pop(), Push(40), Push(50), Push(60), isEmpty(), Push(70)

Learn the concept before tracing it

A data structure is a rule for organising stored values and the operations allowed on them. A *stack* works like a pile of plates: a new plate goes on top, and the next plate removed is the top one. This rule is called **last in, first out (LIFO)**. It does not mean values are numerically sorted.

An array is a row of numbered memory cells. Capacity 5 gives indices 0,1,2,3,4. The variable *top* stores the index of the last occupied cell, so the number of stack items is $top + 1$. Empty means $top = -1$, since index 0 has not yet been filled. A full stack has $top = 4$. The arrows below show which end can change.



Push stores a new item, **pop** removes and returns the last item, **peek** only reads it, and **isEmpty** returns a Boolean answer (true/false). Overflow means trying to push onto a full stack; underflow means trying to pop or peek when empty.

Algorithm: PUSH on an array stack

Step 0: Start.
Step 1: If $top = C - 1$, report overflow and go to Step N.
Step 2: Increase top by 1, store x in $S[top]$, and report success.
Step N: End.

Algorithm: POP on an array stack

Step 0: Start.
Step 1: If $top = -1$, report underflow and go to Step N.
Step 2: Save $S[top]$, decrease top by 1, and report the saved value.
Step N: End.

For peek, first test $top = -1$; if empty report underflow, otherwise return $S[top]$ with no update. For isEmpty, return the result of the test $top = -1$. In the following table the rightmost listed item is the top; every row uses the state left by the preceding row.

A stack is LIFO: push inserts at its top; pop removes and returns the top; peek returns it without changing it. The top index starts at -1; overflow occurs when it is 4 before a push. Related explanation and method: page 3.

Step	Operation	Stack after operation	top	Output / status
0	Initialise	[ ]	-1	Empty
1	Push(10)	[10]	0	Success
2	Push(20)	[10, 20]	1	Success
3	Push(30)	[10, 20, 30]	2	Success
4	Peek()	[10, 20, 30]	2	30
5	Pop()	[10, 20]	1	30
6	Push(40)	[10, 20, 40]	2	Success
7	Push(50)	[10, 20, 40, 50]	3	Success
8	Push(60)	[10, 20, 40, 50, 60]	4	Success; now full
9	isEmpty()	[10, 20, 40, 50, 60]	4	false
10	Push(70)	[10, 20, 40, 50, 60]	4	Overflow

The earlier pop freed one cell, so Push(60) succeeds. Push(70) is the first overflowing operation. A failed push leaves the contents and top unchanged. Peek did not remove 30; pop did. All listed operations take constant time.

Q1(b): Learn selection sort and insertion sort

Full question Q1(b) [Q1 total: 20 marks]. Show all intermediate passes using selection and insertion sort for the given set of elements: 4 3 1 5 2

Sorting puts values in a chosen order. Ascending means smallest to largest. A *pass* is one iteration of the outer loop, even when that iteration makes no change.

Selection sort: divide the array into a finished prefix and an unfinished suffix. Scan the entire suffix, remember its smallest value, and swap that value into the next finished position. On the first pass here, compare 4 with 3 (new minimum 3), then 1 (new minimum 1), then 5 and 2 (neither beats 1). Swap 4 and 1. Do not stop the scan when the first smaller value is found.

Algorithm: Selection sort in ascending order

Step 0: Start.
Step 1: Read array A of length n. Set $i = 0$.
Step 2: If $i \geq n - 1$, report the sorted array and go to Step N.
Step 3: Find the index m of the smallest value in positions i through $n - 1$.
Step 4: Exchange the values at indices i and m unless those indices are equal.
Step 5: Increase i by 1 and go to Step 2.
Step N: End.

Insertion sort: imagine sorting cards in your hand. Keep the left part sorted. Save the next value as *key*, move every larger left-side value one cell right, and put *key* into the gap. Save the key before shifting, since its old cell may be overwritten. The prefix is internally sorted but its values need not yet occupy their final global positions.

Algorithm: Insertion sort in ascending order

Step 0: Start.
Step 1: Read array A of length n. Set $i = 1$.
Step 2: If $i \geq n$, report the sorted array and go to Step N.
Step 3: Save $A[i]$ as key and set $j = i - 1$.
Step 4: While $j \geq 0$ and $A[j] > key$, move $A[j]$ one place right and decrease j by 1.
Step 5: Write key into $A[j + 1]$, increase i by 1, and go to Step 2.
Step N: End.

Selection sort repeatedly finds the minimum of the unsorted suffix. Insertion sort instead inserts the next item into an already sorted prefix. Related explanation and method: page 4.

Pass	Selection: chosen minimum	Swap	Array after pass
0	—	—	$4, 3, 1, 5, 2$
1	1 at index 2	4 and 1	$1, 3, 4, 5, 2$
2	2 at index 4	3 and 2	$1, 2, 4, 5, 3$
3	3 at index 4	4 and 3	$1, 2, 3, 5, 4$
4	4 at index 4	5 and 4	$1, 2, 3, 4, 5$

Pass	Insertion key	Values shifted right	Array after pass
1	3	4	$3, 4, 1, 5, 2$
2	1	4, then 3	$1, 3, 4, 5, 2$
3	5	None	$1, 3, 4, 5, 2$
4	2	5, then 4, then 3	$1, 2, 3, 4, 5$

Selection makes $4 + 3 + 2 + 1 = 10$ key comparisons. Insertion makes $1 + 2 + 0 + 3 = 6$ shifts here. Both use $\Theta(1)$ auxiliary space; both have quadratic worst-case time. Insertion can be linear when already sorted.

Q1(c): BST construction; Q1(d): choosing a structure

Full question Q1(c) [Q1 total: 20 marks]. Create a binary search tree with the given elements: 98, 2, 48, 12, 56, 32, 4, 67, 23, 87, 23, 55, 46.

A *tree* is a hierarchy of nodes joined by parent–child links. Its top node is the root; a node without children is a leaf. A binary tree has separate left and right child positions. A **binary search tree (BST)** adds a rule: every key anywhere in the left subtree is smaller than the current key; every key anywhere in the right subtree is larger. Inserting a key means following these comparisons until an empty child position is reached. It does not mean placing keys in consecutive rows of an array.

Algorithm: Insert a key into a distinct-key BST

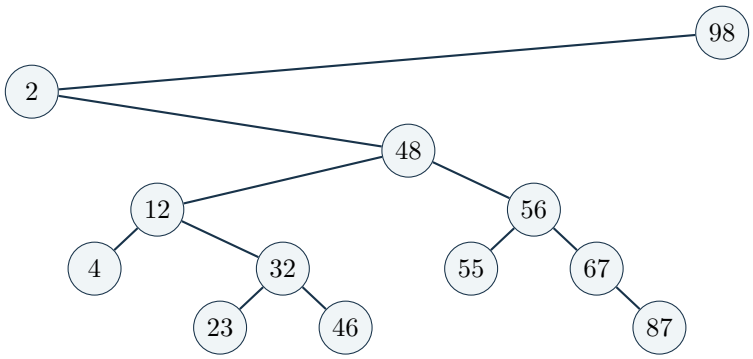
Step 0: Start.
Step 1: If the supplied subtree is empty, allocate a leaf for the key, give it null children, and give its address to the caller; go to Step N. Report allocation failure if no node can be obtained.
Step 2: If the key is smaller than the root key, apply this algorithm to the left subtree and reconnect the resulting left link.
Step 3: If the key is larger, do the same for the right subtree. If equal, leave the tree unchanged under the stated duplicate policy.
Step 4: Give the updated subtree root to the caller; the outermost caller updates the tree root.
Step N: End.

Q1(c): insert $98, 2, 48, 12, 56, 32, 4, 67, 23, 87, 23, 55, 46$

The input repeats 23. We use set semantics: the second 23 is ignored. Each row below describes the new attachment; the complete comparison path is given explicitly.

Key	Comparisons, in the order performed	Result
98	Empty tree	Create root
2	$2 < 98$	Left of 98
48	$48 < 98;\ 48 > 2$	Right of 2
12	$12 < 98;\ 12 > 2;\ 12 < 48$	Left of 48
56	$56 < 98;\ 56 > 2;\ 56 > 48$	Right of 48
32	$32 < 98;\ 32 > 2;\ 32 < 48;\ 32 > 12$	Right of 12
4	$4 < 98;\ 4 > 2;\ 4 < 48;\ 4 < 12$	Left of 12
67	$67 < 98;\ 67 > 2;\ 67 > 48;\ 67 > 56$	Right of 56
23	$23 < 98;\ 23 > 2;\ 23 < 48;\ 23 > 12;\ 23 < 32$	Left of 32
87	$87 < 98;\ 87 > 2;\ 87 > 48;\ 87 > 56;\ 87 > 67$	Right of 67
23 again	$23 < 98;\ 23 > 2;\ 23 < 48;\ 23 > 12;\ 23 < 32;\ 23 = 23$	Ignore duplicate
55	$55 < 98;\ 55 > 2;\ 55 > 48;\ 55 < 56$	Left of 56
46	$46 < 98;\ 46 > 2;\ 46 < 48;\ 46 > 12;\ 46 > 32$	Right of 32

For example, 46 follows $98 \to 2 \to 48 \to 12 \to 32$ and becomes 32's right child, since $32 < 46 < 48$.



Inorder check: $2, 4, 12, 23, 32, 46, 48, 55, 56, 67, 87, 98$. This is increasing and contains all 12 distinct keys. If duplicates are instead represented by a count, annotate the 23 node with count 2. If equal keys are routed right, the extra 23 would become the right child of the first 23; that alternative requires declaring its policy.

Cost. Each insertion is $O(h+1)$; building an unbalanced BST has $O(n^2)$ worst-case total time. Related explanation and method: page 5.

Q1(d): Match operations to a data structure

Full question Q1(d) [Q1 total: 20 marks]. Justify which data structure is most suitable for the given scenario-

1. A system needs to store passenger records with unique ticket IDs. The system must allow constant-time lookup to check whether a passenger exists.
2. An operating system must schedule processes such that the process with minimum running time is scheduled first.

The correct choice depends on the operation we need to make fast. Searching for a particular ID is a different task from repeatedly finding the smallest running time.

A *hash table* uses a function to turn a key into an array index. Different keys can produce the same index: this is a collision, handled by a collision-resolution rule. A *priority queue* removes the item with the best priority; a min-priority queue removes the smallest key. One implementation is a min-heap, a complete binary tree in which a parent's key is no greater than either child's. The root is therefore a minimum, but the whole tree is not a sorted sequence.

Q1(d)(1): passenger records with unique ticket IDs

Choose a **hash table** mapping each ticket ID to its passenger record. Compute the hash index and resolve collisions while comparing complete IDs. Under a suitable hash distribution and controlled load

factor, lookup is expected $O(1)$; insertion and deletion are also expected $O(1)$. The ID uniqueness makes it a natural key.

Key idea. The prompt's constant-time lookup is normally interpreted as expected constant time. A general hash table can take linear time in the worst case. Direct addressing gives worst-case constant lookup only if the ID universe is sufficiently small and storage is affordable.

Q1(d)(2): schedule the minimum-running-time process first

Choose a **min-priority queue**, typically implemented by a binary min-heap keyed by the estimated running time. Read the minimum at the root in $O(1)$ and extract it in $O(\log n)$; insert a new process in $O(\log n)$. Repeating extract-min implements shortest-job-first among currently available jobs. Break equal-time ties by arrival order if required. Related explanation and method: page 6.

Q2: Complexity and a parenthesis checker

Full question Q2(a) [10 marks]. Explain time and space complexity. Explain various asymptotic notations used to calculate time complexity.

Q2(a): time, space, and asymptotic notations

Let n mean the amount of input, such as the number of array items. To predict how an algorithm scales, count simple operations rather than seconds. A loop that processes each item once does about twice the work when the input doubles. Two full nested loops process $n \times n$ pairs, so doubling the input gives about four times the work. A loop that halves the search region needs only one extra iteration when the input doubles.

Time complexity measures elementary-operation growth with input size. Space complexity measures storage growth; auxiliary space excludes the existing input. Hardware speed affects measured seconds, whereas complexity describes scaling.

For positive $g(n)$ and sufficiently large n:

- $T(n) \in O(g(n))$ if $T(n) \leq cg(n)$ for some constant $c > 0$: upper bound.
- $T(n) \in \Omega(g(n))$ if $T(n) \geq cg(n)$ for some $c > 0$: lower bound.
- $T(n) \in \Theta(g(n))$ if both bounds hold, possibly with different constants.
- $T(n) \in o(g(n))$ if $T(n)/g(n) \to 0$; $T(n) \in \omega(g(n))$ if the ratio tends to infinity.

Examples: a single array scan has $T(n) = an + b = \Theta(n)$; two full nested scans are $\Theta(n^2)$; binary search is $O(\log n)$ worst case. Constants and lower-order terms disappear from tight asymptotic growth: $4n^2 + 3n + 8 = \Theta(n^2)$. Keeping only a sum/index needs $\Theta(1)$ auxiliary space, copying n items needs $\Theta(n)$, and recursive call frames also count. Related explanation and method: page 7.

How to simplify a count. If the count is $4n^2 + 3n + 8$, divide by n^2: $4 + 3/n + 8/n^2$ approaches 4. For $n \geq 1$, $4n^2 \leq 4n^2 + 3n + 8 \leq 15n^2$, so the count is bounded above and below by constant multiples of n^2. That is why the tight bound is $\Theta(n^2)$. Big-O is an upper bound, not a synonym for worst case: a best-case function can also have a Big-O bound. State both the case being analysed and the bound.

Full question Q2(b) [10 marks]. Write an algorithm for parenthesis checker using stack? Solve given expression for parenthesis check using stack: `(a-c)*(b+c]`

Why a stack checks nesting. In `([ ])`, the opening `[` is more recent than `(`, so it must be matched first. A counter can check the number of openings but cannot distinguish `(]` from `()`. The stack remembers both nesting order and delimiter type.

Q2(b): check the printed expression `(a-c)*(b+c]`

Use a stack for unmatched openings. The expression is transcribed with its closing square bracket; do not silently replace it with a parenthesis.

Algorithm: Check matching parentheses, brackets, and braces

Step 0: Start.
Step 1: Create an empty stack for unmatched opening delimiters.
Step 2: Read tokens from left to right. Push an opening delimiter. Ignore operands and arithmetic operators.
Step 3: At a closing delimiter, if the stack is empty, report not balanced and go to Step N.
Step 4: For a closing delimiter with a nonempty stack, remove its top opening delimiter. If its type does not match the closing one, report not balanced and go to Step N. Continue Step 2 for the remaining tokens.
Step 5: After all tokens, report balanced only if the stack is empty; otherwise report not balanced.
Step N: End.

Token	Action	Stack afterwards	Reason
`(`	Push	`(`	Opening
`a`	Ignore	`(`	Operand
`-`	Ignore	`(`	Arithmetic operator
`c`	Ignore	`(`	Operand
`)`	Match and pop	Empty	Matches `(`
`*`	Ignore	Empty	Arithmetic operator
`(`	Push	`(`	Opening
`b`	Ignore	`(`	Operand
`+`	Ignore	`(`	Arithmetic operator
`c`	Ignore	`(`	Operand
`]`	Pop `(`; reject	Empty	Type mismatch

Answer: not balanced. The last delimiter should match `[`, but the most recent opening is `(`. The failure is a type mismatch, not a shortage of opening symbols.

Cost. For n tokens, $O(n)$ time and $O(n)$ auxiliary space in the worst case. Each opening is pushed once and popped at most once.

Q3: Arrays versus lists; insertion; BST deletion

Full question Q3(a) [10 marks]. Differentiate between linked list and arrays. Write an algorithm to insert a new node after the given node in singly linked list.

Q3(a): compare the representations

A *pointer* stores the address of another node. A singly linked node contains a data field and a `next` pointer. `head` points to the first node; `null` means there is no node. Node addresses need not be adjacent. By following next pointers we obtain the logical sequence.

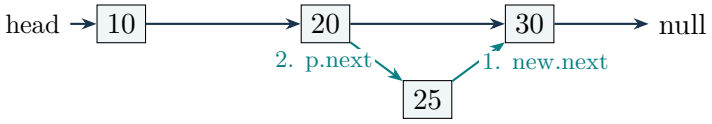
Feature	Array	Singly linked list
Physical storage	Contiguous cells	Separately allocated nodes
Read item at index i	$O(1)$	$O(n)$: follow links
Insert after a known position/node	$O(n)$ shifts in general	$O(1)$ relinking
Find an unsorted value	$O(n)$	$O(n)$
Extra representation storage	Capacity may be unused	One next pointer per node

An array has contiguous storage and constant-time indexed access, but middle insertion/deletion shifts a suffix. A linked list has separately allocated nodes with link overhead; indexed access requires a traversal, but relinking at a known node is constant time. Both need $O(n)$ to search an unsorted collection. A list's size grows by node allocation; an array's capacity must be managed separately. Arrays usually have better locality.

Algorithm: Insert a new node after a supplied node p

Step 0: Start.
Step 1: If p is null, report not found and go to Step N.
Step 2: Allocate a node q holding x. If allocation fails, report failure and go to Step N.
Step 3: Set the next link of q to the old next link of p.
Step 4: Set the next link of p to q; report success.
Step N: End.

Example: $10 \to 20 \to 30 \to null$, with p pointing to 20. Set the new 25 node's next link to 30, then set 20's next to 25. Result $10 \to 20 \to 25 \to 30 \to null$.



After: $10 \to 20 \to 25 \to 30 \to null$.
Keep the old successor before replacing a link.

The assignment order matters: first make 25 point to the old successor 30, then make 20 point to 25. If we overwrite 20.next first without saving the old successor, the remaining chain can become unreachable. At the tail, the same steps set the new next link to `null`. Cost is $\Theta(1)$ once p is supplied; finding p by value is $O(n)$.

Q3(b): Delete a key without breaking BST order

Full question Q3(b) [10 marks]. Explain how deletion is performed in a Binary Search Tree (BST). Write an algorithm to delete a node with a given key from a Binary Search Tree.

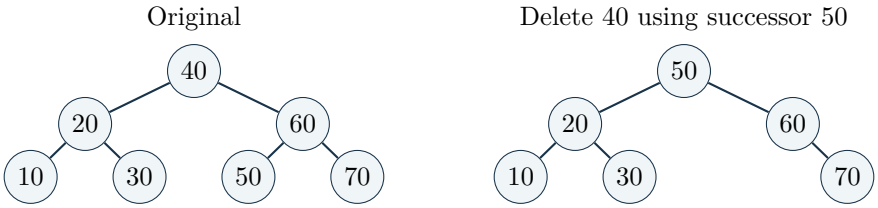
Search down the appropriate side. Leaf: remove it. One child: replace the node by its child. Two children: copy the smallest right-subtree key (successor), then delete that successor.

Algorithm: Delete a key from a BST

Step 0: Start.
Step 1: If the supplied subtree is empty, give null to the caller and go to Step N.
Step 2: If the key is smaller or larger than the root key, apply this algorithm to the corresponding child, reconnect that child, and go to Step 6.
Step 3: If the matching node has no left child, replace it by its right child (possibly null), release the old node, and go to Step 6.
Step 4: If it has no right child, replace it by its left child, release the old node, and go to Step 6.
Step 5: Otherwise find the leftmost node in the right subtree (the successor). Copy its key and record into the matching node; delete that successor from the right subtree with this algorithm and reconnect the right link.
Step 6: Give the resulting subtree root to the caller.
Step N: End.

Use the original BST with root 40, left subtree $20(10, 30)$ and right subtree $60(50, 70)$:

1. Delete leaf 10: set 20's left link to `null`.
2. Then delete 20: its only child is 30, so 40's left link becomes 30.
3. Separately, delete root 40 from the original tree: successor is 50. Replace root key 40 by 50, then delete the original 50 from under 60. Inorder becomes $10, 20, 30, 50, 60, 70$.



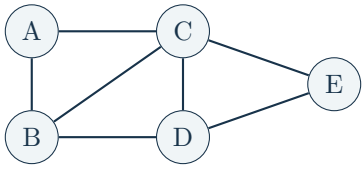
Why choose 50? It is the smallest key in the original right subtree: go right once from 40 to 60, then go left to 50. Every left-subtree key remains smaller than 50, and every remaining right-subtree key is larger. Copying an arbitrary right-subtree key, such as 70, would leave 60 on the wrong side of the new root. Remove the original successor so the key occurs only once.

If the key is absent, the null base case returns without altering any existing link. Copy the full payload associated with a key when records contain more than a number.

Cost. $O(h+1)$ time and recursive space: $O(\log n)$ for balanced shape, $O(n)$ in the worst unbalanced case.

Q4: DFS from A and merge sort

Full question Q4(a) [10 marks]. Write algorithm for performing depth-first search on a graph data structure. Perform a DFS traversal starting from vertex A. Show all intermediate steps and write the final DFS order.



Q4(a): graph and deterministic neighbour order

A graph consists of vertices (objects) and edges (connections). Here the edges have no arrowheads, so they can be followed both ways. **Depth-first search (DFS)** follows one unfinished branch as deeply as possible. When it cannot reach an unvisited neighbour, it returns to the vertex that called it: this is backtracking. A visited flag prevents endless cycling along edges such as A–B–C–A. The recursive call stack remembers the active branch.

Algorithm: Depth-first search from a source vertex

Step 0: Start.
Step 1: Mark every vertex unvisited. Apply the following visit procedure to the source.
Step 2: For the current vertex u, mark it visited and record it.
Step 3: Examine its neighbours in the stated order. For each still-unvisited neighbour, apply the visit procedure of Steps 2–3 to that neighbour before examining the next one.
Step 4: When a call has no unvisited neighbour left, return to its caller. When the source call finishes, report the recorded traversal.
Step N: End.

The undirected edges are AB, AC, BC, BD, CD, CE, DE. Use alphabetical neighbours: A:B,C; B:A,C,D; C:A,B,D,E; D:B,C,E; E:C,D. Apply DFS from page 10.

Step	Action	Active recursive path	Output
1	Visit A	A	A
2	A chooses B	A,B	A B
3	B skips A, chooses C	A,B,C	A B C
4	C skips A,B, chooses D	A,B,C,D	A B C D
5	D skips B,C, chooses E	A,B,C,D,E	A B C D E
6	E has no unvisited neighbour	Backtrack E,D,C,B,A	Unchanged

DFS order: A, B, C, D, E. Backtracking does not print vertices again. Other neighbour orders can produce different valid answers. With adjacency lists the general cost is $O(V+E)$ time, $O(V)$ auxiliary space.

Q4(b): Merge sort from the first split to the final merge

Full question Q4(b) [10 marks]. Write algorithm for performing Merge sort. Show how the following set of elements will be sorted using Merge sort: 45 21 67 30 19 55 20 70.

Merge sort uses **divide and conquer**: split a problem into smaller versions, solve them, and combine their answers. A one-item array is already sorted. The merge operation works only because its two input runs are already sorted. Compare their first remaining values, copy the smaller, and advance just that run. Once a run is empty, copy the rest of the other run without further comparisons.

Algorithm: MERGE SORT on inclusive indices $lo, \ldots, hi$

Step 0: Start.
Step 1: If $lo \geq hi$, the segment needs no change; go to Step N.
Step 2: Set $mid = lo + \lfloor (hi - lo)/2 \rfloor$.
Step 3: Apply this algorithm to $lo, \ldots, mid$, then to $mid + 1, \ldots, hi$.
Step 4: Combine the two sorted halves with the MERGE algorithm below.
Step N: End.

Algorithm: MERGE two sorted halves

Step 0: Start.
Step 1: Copy the left half into L and the right half into R. Start at the first item of each; the next output position is lo.
Step 2: While both halves have an unconsumed item, copy the smaller front item to the output. On equality choose from L. Advance that half and the output position.
Step 3: Copy all unconsumed items of the nonempty half in their existing order.
Step N: End.

Q4(b): merge-sort $45, 21, 67, 30, 19, 55, 20, 70$

Use `MERGE_SORT(A,0,7)` and `MERGE` from page 11. Splitting:

$$[45, 21, 67, 30] \mid [19, 55, 20, 70]$$
$$[45, 21] \mid [67, 30] \mid [19, 55] \mid [20, 70]$$
$$[45] \mid [21] \mid [67] \mid [30] \mid [19] \mid [55] \mid [20] \mid [70].$$

Merge singletons: $[21, 45] \mid [30, 67] \mid [19, 55] \mid [20, 70]$.

Merge pairs: $[21, 30, 45, 67] \mid [19, 20, 55, 70]$. For the left merge take 21,30,45, then copy 67; for the right take 19,20,55, then copy 70.

Final-merge comparison	Take	Final-merge comparison	Take
21 vs 19	19	45 vs 55	45
21 vs 20	20	67 vs 55	55
21 vs 55	21	67 vs 70	67
30 vs 55	30	Left exhausted	Copy 70

Result: $[19, 20, 21, 30, 45, 55, 67, 70]$**.** Every value appears once. Time is $\Theta(n \log n)$; auxiliary storage is $O(n)$.

Why $n \log n$**?** For eight values, there are three splitting/merging levels: $8 \to 4 \to 2 \to 1$. Each merge level processes eight values in total. In general, there are about $\log_2 n$ levels and linear work per level, giving $\Theta(n \log n)$. The temporary arrays are the reason the standard array version needs $O(n)$ extra storage. Taking the left item on equality preserves the order of equal records (stability).

Q5: Infix to postfix; a BST from traversals

Full question Q5(a) [10 marks]. Write algorithm to convert Infix notation to Postfix notation using stack. Apply the same on following expression: `(A - 2 * (B + C) / D * E) + F`

Q5(a): $(A - 2 * (B + C)/D * E) + F$

An *operand* is a number or variable, such as 2 or A. An *operator*, such as + or /, combines operands. Infix puts the operator between its operands: $A + B$. Postfix puts it after them: `A B +`. Prefix (Polish) puts it first: `+ A B`. In postfix the operand order is retained; we rearrange operators so their required operands appear first.

Precedence decides which operation binds first: parentheses, then exponentiation, then multiplication/division, then addition/subtraction. **Associativity** breaks equal-precedence ties. Multiplication and division are left associative, so $x/y*z = (x/y)*z$. Exponentiation is right associative, so $x^{y^z} = x^{(y^z)}$.

We use two separate places: an *output sequence* that is already final, and a *stack of pending operators*. Send operands directly to output. Delay an operator until both its operand expressions have been output. An opening parenthesis is a barrier; operators inside it must finish before it is discarded.

Algorithm: Convert a valid infix expression to postfix

Step 0: Start.
Step 1: Create an empty output sequence and an empty operator stack. Read one token at a time from left to right.
Step 2: For an operand, append it to the output. For an opening parenthesis, push it.
Step 3: For a closing parenthesis, pop operators into the output until the opening parenthesis is reached; discard that opening. If none exists, report invalid expression and go to Step N.
Step 4: For an operator, pop stack operators of higher precedence, or equal precedence when the incoming operator is left associative. Stop at an opening parenthesis or an operator that does not meet this rule. Push the incoming operator.
Step 5: Repeat Steps 2–4 until all tokens are processed.
Step 6: Pop remaining operators into the output. If a parenthesis remains, report invalid expression; otherwise report the postfix sequence.
Step N: End.

Use the operator-stack algorithm on page 12. Multiplication and division have equal precedence and associate left, so pop the existing `*` before pushing `/`, and pop `/` before the following `*`. Stack is bottom to top.

Token	Operator stack after token	Output after token
`(`	`(`	—
A	`(`	A
`-`	`( -`	A
2	`( -`	A 2
`*`	`( - *`	A 2
`(`	`( - * (`	A 2
B	`( - * (`	A 2 B
`+`	`( - * ( +`	A 2 B
C	`( - * ( +`	A 2 B C
`)`	`( - *`	A 2 B C +
`/`	`( - /`	A 2 B C + *
D	`( - /`	A 2 B C + * D
`*`	`( - *`	A 2 B C + * D /
E	`( - *`	A 2 B C + * D / E
`)`	Empty	A 2 B C + * D / E * -
`+`	`+`	A 2 B C + * D / E * -
F	`+`	A 2 B C + * D / E * - F
End	Empty	A 2 B C + * D / E * - F +

Postfix: `A 2 B C + * D / E * - F +`. Its grouping is $A - (((2(B + C))/D)E) + F$. Conversion takes $O(n)$ time and space.

Read the difficult rows. At the first `)`, append + to finish `B C +`, then discard the opening bracket. At /, the existing * has equal precedence and must execute first, so output it before stacking /. At the next *, output / for the same reason. At the outer `)`, output * then -, finishing the parenthesised subtraction. The final + waits for F and is emitted last.

Q5(b): Build the BST and verify both traversals

Full question Q5(b) [10 marks]. Write an algorithm to insert new node into a binary search tree. Construct binary search tree from the given traversal:

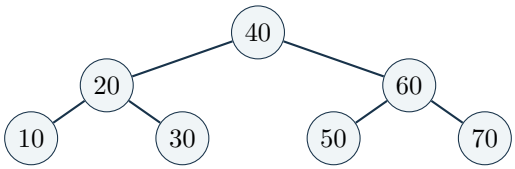
Preorder: 40, 20, 10, 30, 60, 50, 70

Postorder: 10, 30, 20, 50, 70, 60, 40

Traversal means visiting every node in a specified order. Preorder visits root, left subtree, right subtree. Postorder visits left subtree, right subtree, root. The rules apply recursively inside each subtree, not just to the top level.

Use `root=BST_INSERT(root,key)` from page 5. Compare to each node, go left for a smaller key and right for larger, and attach a leaf at the first null position.

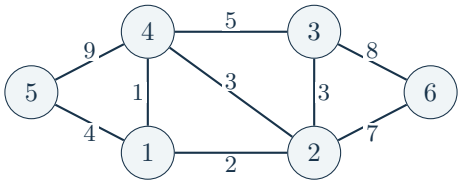
Preorder: $40, 20, 10, 30, 60, 50, 70$. Root is 40; smaller keys form its left subtree and larger keys form its right subtree. Left preorder $20, 10, 30$ gives root 20 with children 10 and 30. Right preorder $60, 50, 70$ gives root 60 with children 50 and 70.



Postorder check: $10, 30, 20, 50, 70, 60, 40$, exactly as supplied. Inorder is $10, 20, 30, 40, 50, 60, 70$. Preorder/postorder alone are ambiguous for general binary trees, but the stated distinct-key BST requirement resolves this construction.

Q6: Prim from vertex 5; deleting from a linked list

Full question Q6(a) [10 marks]. What is minimum spanning tree. Construct a minimum spanning tree of the given graph using Prim's algorithm. Start from vertex 5. Show all intermediate steps.



Q6(a): MST definition and graph

Spanning means all vertices are included. **Tree** means connected without cycles. With V vertices, a spanning tree has exactly $V - 1$ edges. **Minimum** means the sum of its selected edge weights is as small as possible; it does not mean that every individual journey from a source is shortest.

Prim grows one connected set S. A *crossing edge* has one endpoint in S and one outside. Only these edges can add a new vertex without closing a cycle. At each step choose a cheapest crossing edge, even if a cheaper edge exists elsewhere in the graph.

Algorithm: Prim starting at the specified vertex

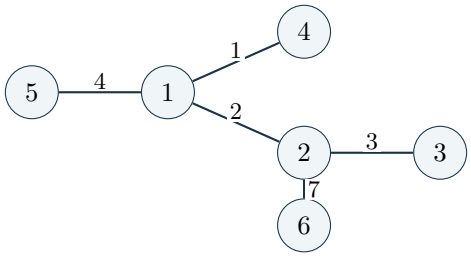
Step 0: Start.
Step 1: Put only the starting vertex in set S. Begin an empty selected-edge set and cost 0.
Step 2: If all vertices are in S, report the selected edges and cost; go to Step N.
Step 3: Choose a cheapest edge with exactly one endpoint in S. If none exists, report that the graph is disconnected and go to Step N.
Step 4: Add that edge and its outside endpoint; add its weight to the cost and go to Step 2.
Step N: End.

An MST connects all vertices without cycles with minimum total edge weight. The paper's undirected weighted edges are:

$$(5,4):9,\ (5,1):4,\ (4,1):1,\ (4,3):5,\ (4,2):3,\ (1,2):2,\ (3,2):3,\ (3,6):8,\ (2,6):7.$$

Prim repeatedly chooses the cheapest edge crossing from the included set to an outside vertex. Start with $S = \{5\}$; apply page 13.

Step	Included before	Selected edge	Why / running cost
1	$\{5\}$	$5 - 1 : 4$	Cheaper than $5 - 4 : 9$; cost 4
2	$\{5, 1\}$	$1 - 4 : 1$	Cheapest crossing edge; cost 5
3	$\{5, 1, 4\}$	$1 - 2 : 2$	Beats $4 - 2 : 3$, $4 - 3 : 5$; cost 7
4	$\{5, 1, 4, 2\}$	$2 - 3 : 3$	Beats $4 - 3 : 5$, edges to 6; cost 10
5	$\{5, 1, 4, 2, 3\}$	$2 - 6 : 7$	Beats $3 - 6 : 8$; cost 17



MST cost $= 4 + 1 + 2 + 3 + 7 = 17$**.** There are 6 vertices and 5 edges, all connected. The cheap edge $4 - 2 : 3$ is not chosen after step 3 because both endpoints are then included; it would make a cycle.

Q6(b): Doubly linked lists and deletion by value

Full question Q6(b) [10 marks]. Explain a doubly linked list and describe how it differs from a singly linked list. Write an algorithm to delete a node with a given value from a singly linked list. Consider both cases: when the node is found and when it is not found.

A singly linked node stores data and next. A doubly linked node adds prev, so one can move backwards or delete a known node without searching for its predecessor. The tradeoff is extra memory and more link updates. The requested deletion algorithm is for a *singly* linked list:

Algorithm: Delete the first node with a given value

Step 0: Start.
Step 1: Set current to head and previous to null.
Step 2: Move along next links until current is null or its data equals the key; before each move, set previous to current.
Step 3: If current is null, report not found and go to Step N.
Step 4: If previous is null, move head to current.next; otherwise set previous.next to current.next.
Step 5: Release current; report success and the updated head.
Step N: End.

For $10 \to 20 \to 30 \to \mathit{null}$, deleting 20 sets 10.next to the 30 node. Deleting the head instead updates head. Deleting the tail sets the predecessor's next to null. An empty list or absent value returns NOT_FOUND without changes. This deletes the first occurrence; deleting all matches needs continuing the traversal.

Cost. $O(n)$ time to search, $\Theta(1)$ relinking time and auxiliary space.

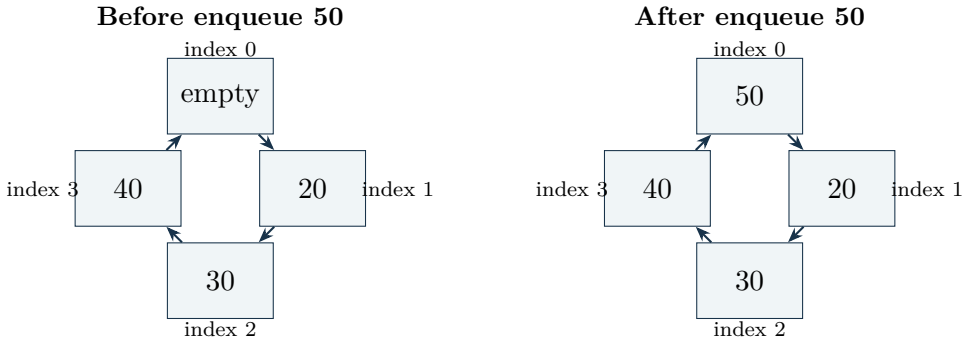
Q7: Circular queues and Huffman coding

Full question Q7(a) [10 marks]. Explain the concept of a Circular Queue, highlighting its need over a linear queue. Write the algorithm for its primary operations: enqueue() and dequeue(), along with the conditions for queue overflow and underflow. Give any two real-world applications of circular queue.

Q7(a): need, operations, overflow, and underflow

A *queue* follows FIFO: first in, first out. Add at the rear, remove at the front. In a circular array, the cell after the last index is index 0. The arithmetic $(i + 1) \bmod C$ gives exactly this wrap: with $C = 3$, indices advance $0, 1, 2, 0, 1, \ldots$. No data item has to move when an index wraps.

Here rear means the *next free insertion slot*, front means the next item to remove, and count is the number currently present. Because front and rear can coincide both when full and when empty, we must use count to tell those states apart.



A circular queue is a FIFO queue whose indices wrap modulo capacity. It reuses cells freed at the beginning, avoiding a linear queue's wasted prefix. In the count-based convention, start $front = rear = count = 0$, with rear pointing to the next free cell.

Algorithm: ENQUEUE in a circular array queue

Step 0: Start.
Step 1: If $count = C$, report overflow and go to Step N.
Step 2: Store x at index rear; replace rear by $(rear + 1) \bmod C$.
Step 3: Increase count by 1 and report success.
Step N: End.

Algorithm: DEQUEUE in a circular array queue

Step 0: Start.
Step 1: If $count = 0$, report underflow and go to Step N.
Step 2: Save the value at index front; replace front by $(front + 1) \bmod C$.
Step 3: Decrease count by 1 and report the saved value.
Step N: End.

Both take $\Theta(1)$ time. For capacity 3, enqueue 10,20,30 gives $front = rear = 0, count = 3$. Dequeue 10 gives $front = 1, count = 2$; enqueue 40 writes slot 0 and gives logical queue $[20, 30, 40]$. Another enqueue overflows. Empty dequeue underflows. Applications: bounded audio/network buffers and round-robin process scheduling. Display procedure and another wrapped example: page 34.

Q7(b): Build a Huffman code one merge at a time

Full question Q7(b) [10 marks]. Write Huffman encoding algorithm.
A text file contains the following letter frequencies:
P:30, Q:3, R:6, S:12, T:34, U:7
Construct the Huffman tree and generate the Huffman codes. Show all intermediate steps.

Compression can give frequent letters short codes and rare letters longer codes. A *prefix-free* code has no complete letter code that is the beginning of another code. Thus a decoder can follow bits from the tree root until it reaches a leaf, output that letter, and restart from the root without separators.

Huffman's algorithm repeatedly combines the two least frequent *current trees*. A merged weight must be put back into consideration. For example, after merging 3 and 6 here, the new 9 is smaller than 12, so it participates in the next merge. The original frequency order alone is no longer enough.

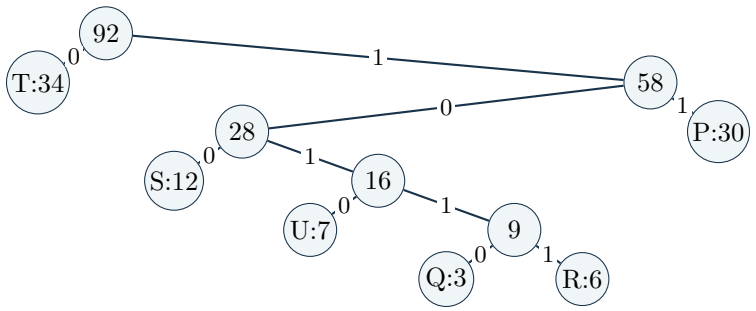
Algorithm: Huffman code construction

Step 0: Start.
Step 1: Create one leaf per symbol with its frequency. Place all leaves in a min-priority queue.
Step 2: If there are no symbols, report an empty code and go to Step N. If there is only one symbol, assign code 0 and go to Step N.
Step 3: Remove the two least-weight trees. Join them under a new parent weighted by their sum; reinsert that parent.
Step 4: Repeat Step 3 until a single tree remains.
Step 5: Label left edges 0 and right edges 1. For each symbol, report the bit string on its root-to-leaf path.
Step N: End.

Q7(b): frequencies P:30, Q:3, R:6, S:12, T:34, U:7

Use a min-priority queue; remove two minimum weights, merge, reinsert, and repeat. Left edge = 0, right edge = 1. Related explanation and method: page 15.

Step	Merge	Remaining weights in increasing order
0	—	$3, 6, 7, 12, 30, 34$
1	$Q3 + R6 = 9$	$7, 9, 12, 30, 34$
2	$U7 + 9 = 16$	$12, 16, 30, 34$
3	$S12 + 16 = 28$	$28, 30, 34$
4	$28 + P30 = 58$	$34, 58$
5	$T34 + 58 = 92$	92



Symbol	P	Q	R	S	T	U
Code	11	10110	10111	100	0	1010
Length	2	5	5	3	1	4

Read a code from the diagram. To reach Q, follow right (1), left (0), right (1), right (1), left (0): code 10110. Its frequency is 3, so Q contributes $3 \times 5 = 15$ encoded bits. T is the root's left leaf, so its single bit 0 is enough. No other code starts with 0.

Weighted cost $= 30(2) + 3(5) + 6(5) + 12(3) + 34(1) + 7(4) = \boxed{203}$ bits. Check: merged weights sum to $9 + 16 + 28 + 58 + 92 = 203$. Average $203/92 \approx 2.207$ bits/symbol. A fixed-length code for 6 symbols takes 3 bits each, or 276 data bits; this code saves 73 data bits, excluding the codebook. All symbols are leaves, so the code is prefix free.

2 Paper B — Re-examination, 14 February 2024

Question 1(a)

Q1: Answer briefly. [20 marks in total]

Full question Q1(a) [5 marks]. What is recursion? Give recursive algorithm to calculate the factorial of the given number.

Q1(a): definition and recursive factorial

Every recursive call has its own local input and its own unfinished work. For $F(3)$, the call with input 3 waits for $F(2)$; the call with input 2 waits for $F(1)$; the call with input 1 waits for $F(0)$. The base case must be reached before any of those pending multiplications can finish. Require an integer $n \geq 0$; factorial in this algorithm is not defined for negative or fractional inputs.

Recursion is a method in which a function solves a problem by calling itself on a smaller instance. A base case ends the chain. For a nonnegative integer, $0! = 1$ and $n! = n(n-1)!$ when $n > 0$.

Algorithm: Recursive factorial of a nonnegative integer n

Step 0: Start.
Step 1: If $n < 0$, report invalid input and go to Step N.
Step 2: If $n = 0$, give the calling operation the value 1 and go to Step N.
Step 3: Otherwise apply this same algorithm to $n - 1$; multiply its result by n and give that product to the caller.
Step N: End.

For 5: $F(5) = 5F(4) = 5(4F(3)) = \cdots = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 \cdot F(0) = 120$. The decreasing argument guarantees termination. There are $n + 1$ calls including the base case: $\Theta(n)$ time and $\Theta(n)$ auxiliary call-stack space under fixed-size arithmetic.

Call made	Work waiting on the stack	Value when it returns
$F(3)$	$3 \times F(2)$	$3 \times 2 = 6$
$F(2)$	$2 \times F(1)$	$2 \times 1 = 2$
$F(1)$	$1 \times F(0)$	$1 \times 1 = 1$
$F(0)$	Base case: no further call	1

Calls are made down the table and return up the table. Returning means giving an answer back to the particular call that was waiting for it; it does not restart the entire algorithm.

Q1(b): Linked memory, sorted marks, deletion, and insertion

Full question Q1(b) [5 marks]. The memory representation of the marks of Chemistry is given below. Empty places shows that these students didn't appear the exam.

Roll Number	1	2	3	4	5	6	7	8	9	10
Marks	50		60	20	30		40		90	35

I. Draw the memory representation of linked list for storing the student's marks in ascending order. [2]
II. Draw the memory representation of linked list if roll number 3 and 7 leaves the program in linked list created in I [2]
III. Draw the memory representation of linked list after allocating Roll number 8 gives the exam later and got 59 marks. [1]

Each node must retain the student's identity and mark, plus the address of the next node. The address is a location, the mark is the sorting key, and the roll number is a record field. These three roles are different. Arrange links by marks rather than by roll number; ignore absent students rather than assigning them an invented zero.

Q1(b)(I): store present students in ascending mark order

The source table contains $(roll, mark)$ pairs $(1, 50), (3, 60), (4, 20), (5, 30), (7, 40), (9, 90), (10, 35)$. Rolls 2,6,8 are absent; no zero-mark nodes should be invented. Store both roll and mark, so sorting does not discard student identity.

Use symbolic addresses R1, R3, etc.; these represent node locations, not consecutive numeric addresses. Head points to R4.

Address	Roll	Mark	next
R4	4	20	R5
R5	5	30	R10
R10	10	35	R7
R7	7	40	R1
R1	1	50	R3
R3	3	60	R9
R9	9	90	null

Thus the logical chain, shown as $(roll : mark)$, is

$$head \to (4 : 20) \to (5 : 30) \to (10 : 35) \to (7 : 40) \to (1 : 50) \to (3 : 60) \to (9 : 90) \to null.$$

Q1(b)(II): rolls 3 and 7 leave

Delete R7 by setting R10.next = R1. Delete R3 by setting R1.next = R9. Free both removed nodes. Head stays R4:

$$head \to (4 : 20) \to (5 : 30) \to (10 : 35) \to (1 : 50) \to (9 : 90) \to null.$$

Q1(b)(III): roll 8 later scores 59

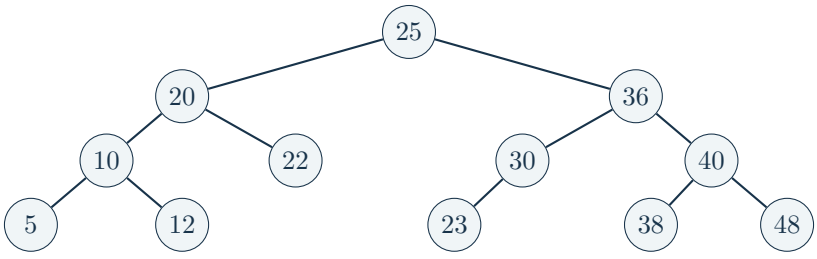
Treat this as a continuation after the departures. Allocate R8 holding $(8, 59)$, set R8.next = R9, then R1.next = R8:

$$head \to (4 : 20) \to (5 : 30) \to (10 : 35) \to (1 : 50) \to (8 : 59) \to (9 : 90) \to null.$$

The result remains sorted since $50 < 59 < 90$. If part III were instead intended independently of part II, insert R8 between R1 and R3 in the original chain; retain R7 and R3. The sequential interpretation is used throughout this guide.

Q1(c): tree terminology; Q1(d): graph representations

Full question Q1(c) [5 marks]. Find the degree of Node 20, Siblings of Node 10, level of the tree, height of Node 36 from the given binary tree.



Q1(c): read the supplied binary tree

A node's *children* are immediately below it; its descendants include all deeper nodes below it. Siblings have the same immediate parent. Tree degree counts children. Depth counts edges from the root, and

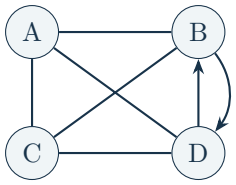
height counts edges on a longest downward path from the node to a leaf. The word "level" is used with both zero-based and one-based conventions, so give the number of occupied levels as well as the convention.

The root is 25; its children are 20 and 36. Node 20 has children 10 and 22; node 10 has 5 and 12. Node 36 has 30 and 40; node 30 has left child 23, and 40 has children 38 and 48.

- **Degree of node 20 = 2**, because it has children 10 and 22.
- **Sibling of node 10 = 22**, because both have parent 20.
- **Tree has 4 levels**: root at depth 0, leaves as deep as depth 3. If numbered from 1, levels run from 1 through 4. Tree height is 3 edges.
- **Height of node 36 = 2 edges**: paths $36 \to 30 \to 23$ and $36 \to 40 \to 38$ have two edges. Under a node-count convention it would be 3; state the convention.

This is a binary-tree terminology question. Do not assume the diagram is a BST: 23 lies in the right subtree of 25 despite being smaller than 25.

Full question Q1(d) [5 marks]. Give the adjacency matrix and adjacency list for the below given graph.



Q1(d): directed adjacency matrix and list

A directed edge is an arrow, so $u \to v$ does not imply $v \to u$. To make a matrix, first choose a fixed vertex order for *both* rows and columns. Scan every arrow once and write 1 at (source row, destination column); fill the other cells with 0. To make a list, collect the destinations of all arrows leaving each source. In an undirected graph each edge would appear in both directions; here it appears only in its printed direction.

The arrows are $A \to B$, $A \to C$, $B \to C$, $B \to D$, $C \to D$, $D \to A$, $D \to B$. In particular, B and D have arrows in both directions. Use order A,B,C,D.

	A	B	C	D
A	0	1	1	0
B	0	0	1	1
C	0	0	0	1
D	1	1	0	0

A: B, C
B: C, D
C: D
D: A, B

Read matrix cell (u, v) as an arrow *from row u to column v*. Thus $M_{DA} = 1$, but $M_{AD} = 0$. The matrix need not be symmetric for directed graphs. The sum of all entries is 7, matching the seven listed arrows.

Cost. A matrix uses $\Theta(V^2)$ space; an adjacency list uses $\Theta(V + E)$. Lists support traversing only actual outgoing neighbours. Related explanation and method: page 10.

Q2: A linear scan, postfix evaluation, and a queue

Full question Q2(a) [10 marks].

i. Define time and space complexity and find the time complexity of given algorithm

```
Sum (A, n)
{ s = 0;
  for(i=0; i<n; i++) { s = s + A[i]; }
  return s; }
```

ii. Write an algorithm to evaluate postfix expression using stack and apply the algorithm on given postfix expression to evaluate it.

```
9 3 4 * 8 + 4 / -
```

Q2(a)(i): complexity of the supplied sum algorithm

Time complexity is how the number of operations grows with input size. Space complexity is how storage grows; *auxiliary* space excludes the input itself. In the printed loop, i takes the values $0, 1, \ldots, n-1$. Thus there are n additions to the sum and n increments; the loop test is evaluated $n+1$ times, including its final failure. Initialisation and return contribute only a fixed number of operations.

The function initialises $s = 0$, adds each of $A[0], \ldots, A[n-1]$ to s, then returns it. The loop executes exactly n times, regardless of the values in the array. Consequently $T(n) = an + b = \Theta(n)$ in best and worst cases under unit-cost arithmetic. Auxiliary space is $\Theta(1)$ for s and the loop index; total space including the array is $\Theta(n)$. Definitions and formal bounds are in page 7.

Q2(a)(ii): evaluate `9 3 4 * 8 + 4 / -`

Postfix has no parentheses because an operator follows the subexpressions it combines. The stack stores *completed values*, unlike conversion, where the stack stores *pending operators*. On reading an operator, the top is the right operand because it arrived most recently. For subtraction/division the order is crucial: if the stack is $[9, 5]$, popping gives right=5 then left=9, so compute $9 - 5$, not $5 - 9$.

Algorithm: Evaluate a postfix expression

Step 0: Start.
Step 1: Create an empty value stack and read tokens from left to right.
Step 2: Push the numerical value of each operand.
Step 3: For a binary operator, require at least two values. Remove the right operand first, then the left; calculate left operator right and push the result. Report invalid input if an operand is missing or the arithmetic is undefined.
Step 4: Repeat Steps 2–3 for all tokens. If an error occurs, go to Step N.
Step 5: If exactly one value remains, report it; otherwise report an invalid expression.
Step N: End.

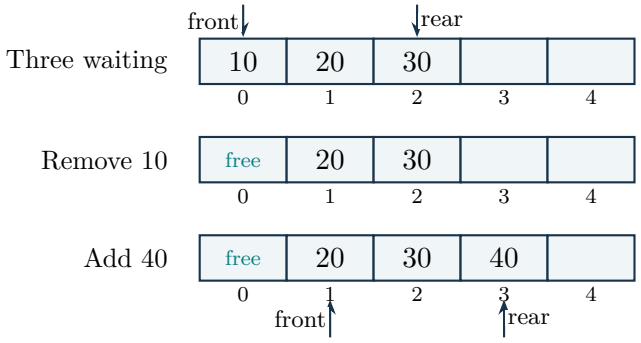
Check division by zero and other invalid operations when applying an operator.

Token	Action	Stack afterwards
9	Push	$[9]$
3	Push	$[9, 3]$
4	Push	$[9, 3, 4]$
*	$3 \times 4 = 12$	$[9, 12]$
8	Push	$[9, 12, 8]$
+	$12 + 8 = 20$	$[9, 20]$
4	Push	$[9, 20, 4]$
/	$20/4 = 5$	$[9, 5]$
-	$9 - 5 = 4$	$[4]$

Answer: 4. Equivalent infix is $9 - ((3 \cdot 4 + 8)/4)$. Pop the right operand first. General cost is $O(n)$ time and stack space.

Q2(b): A FIFO queue with insertion and removal

Full question Q2(b) [10 marks]. Explain Queue as a data structure and its application using any real life example? Write an algorithm or function to insert and delete element in queue.



Removal is at front.
Insertion is at rear.
Service order after both:
$20 \to 30 \to 40$.

Q2(b): FIFO queue and insertion/deletion

A queue serves the earliest inserted item first, as in a printer spooler. For a linear array queue initialise $front = 0, rear = -1$:

Algorithm: ENQUEUE in a linear array queue

Step 0: Start.
Step 1: If $rear = C - 1$, report overflow and go to Step N.
Step 2: Increase rear by 1 and store x in $Q[rear]$.
Step N: End.

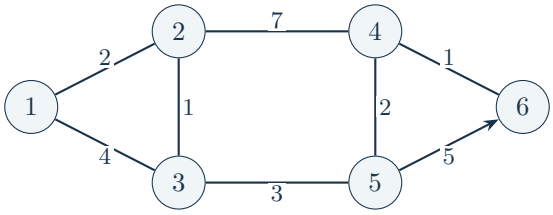
Algorithm: DEQUEUE in a linear array queue

Step 0: Start.
Step 1: If $front > rear$, report underflow and go to Step N.
Step 2: Save $Q[front]$ and increase front by 1.
Step 3: If the queue is now empty, reset $front = 0$ and $rear = -1$. Report the saved value.
Step N: End.

Enqueue 10 then 20 gives $[10, 20]$; dequeue returns 10 and leaves $[20]$. Each operation is $\Theta(1)$, but the nonempty linear queue can waste freed prefix slots. A circular queue (page 14) reuses them.

Q3: Dijkstra from 1; infix to postfix

Full question Q3(a) [10 marks]. Explain the steps in Dijkstra's algorithm. Find the shortest path from vertex 1 to remaining all vertices for the graph given below using Dijkstra's Algorithm.



Q3(a): directed shortest paths

The length of a weighted path is the sum of its edge weights. Dijkstra finds the least such sum from one source to every reachable vertex, provided all edge weights are nonnegative. It does not count only the number of edges.

A *tentative distance* is the best path length discovered so far. Infinity means no path has yet been found, not a very large usable number. A *finalised* vertex has a proven shortest distance. At each iteration select the unfinalised vertex with minimum tentative distance, then try to improve all its outgoing neighbours. The improvement test is called *relaxation*:

$$d[u] + w(u, v) < d[v] \quad \Longrightarrow \quad d[v] \leftarrow d[u] + w(u, v), \quad parent[v] \leftarrow u.$$

Algorithm: Dijkstra with nonnegative edge weights

Step 0: Start.
Step 1: Give the source distance 0 and every other vertex distance ∞; give all vertices a null parent and mark all unfinalised.
Step 2: Choose an unfinalised vertex u with smallest tentative distance. If none exists or that distance is ∞, report all distances and parents and go to Step N.
Step 3: Mark u finalised. For each outgoing edge $u \to v$ to an unfinalised vertex, compare $d[u] + w(u, v)$ with $d[v]$. If smaller, replace $d[v]$ and record parent u.
Step 4: Go to Step 2. After termination, reconstruct a path by following parents backwards and reversing that sequence.
Step N: End.

Edges: $1 \to 2 : 2$, $1 \to 3 : 4$, $2 \to 3 : 1$, $2 \to 4 : 7$, $3 \to 5 : 3$, $5 \to 4 : 2$, $4 \to 6 : 1$, $5 \to 6 : 5$. Initialise source 1 to 0 and others to infinity. Repeatedly finalise the least tentative distance and relax all outgoing edges. Related explanation and method: page 21.

Finalised now	1	2	3	4	5	6
Initial	0	∞	∞	∞	∞	∞
1	0	2	4	∞	∞	∞
2	0	2	3	9	∞	∞
3	0	2	3	9	6	∞
5	0	2	3	8	6	11
4	0	2	3	8	6	9
6	0	2	3	8	6	9

Every relaxation, including the initial discoveries.

1. Finalise 1 at 0. Edge $1 \to 2$ proposes $0 + 2 = 2$; edge $1 \to 3$ proposes $0 + 4 = 4$. Both improve infinity.
2. Finalise 2 at 2, since 2 is the least tentative distance. Edge $2 \to 3$ proposes $2 + 1 = 3 < 4$, so change 3's parent from 1 to 2. Edge $2 \to 4$ proposes $2 + 7 = 9$.
3. Finalise 3 at 3. Edge $3 \to 5$ proposes $3 + 3 = 6$.
4. Finalise 5 at 6 before 4 at 9. Edge $5 \to 4$ proposes $6 + 2 = 8 < 9$, so 4's parent becomes 5. Edge $5 \to 6$ proposes $6 + 5 = 11$.
5. Finalise 4 at 8. Edge $4 \to 6$ proposes $8 + 1 = 9 < 11$, so 6's parent becomes 4.
6. Finalise 6 at 9. It has no outgoing edge; all reachable vertices are finished.

To recover the path to 6, follow parents backwards: $6 \leftarrow 4 \leftarrow 5 \leftarrow 3 \leftarrow 2 \leftarrow 1$. Reverse it for the forward route. A vertex's first discovered distance may improve later; it becomes permanent only at finalisation.

The simple implementation scans vertices for the minimum, taking $O(V^2 + E)$ time and $O(V)$ auxiliary storage. With adjacency lists and a binary heap it takes $O((V + E) \log V)$ time. Nonnegative weights make finalisation safe: a later detour through an equally distant or farther frontier cannot reduce the selected minimum.

Updates: d_3 improves from 4 to $2 + 1 = 3$; $d_5 = 3 + 3 = 6$; d_4 improves from 9 to $6 + 2 = 8$; d_6 improves from $6 + 5 = 11$ to $8 + 1 = 9$. Final paths: The path to 6 is $1 \to 2 \to 3 \to 5 \to 4 \to 6$, cost 9. Its successive prefixes give the shortest paths to 2 (cost 2), 3 (3), 5 (6), and 4 (8). The source has distance 0.

Q3(b): Infix, postfix, prefix, and the operator stack

Full question Q3(b) [10 marks]. Define infix, postfix, and prefix expression? Write an algorithm to convert infix expression to postfix expression. Convert given infix expression in to postfix expression.
`A+(B*C(D/E^F)*G)*H`

Infix: `A+B`; prefix: `+ A B`; postfix: `A B +`. The scan prints `A+(B*C(D/EF̂)*G)*H`. It lacks an explicit operator between C and the following parenthesis. Interpreting this mathematical juxtaposition as

multiplication gives $A+(B*C*(D/E^F)*G)*H$. The operator-stack algorithm, including right-associative exponentiation, is in page 11. Trace of the interpreted expression:

Token	Stack	Output
A	Empty	A
+	+	A
(	+ (	A
B	+ (	A B
*	+ (*	A B
C	+ (*	A B C
*	+ (*	A B C *
(	+ (* (	A B C *
D	+ (* (	A B C * D
/	+ (* (/	A B C * D
E	+ (* (/	A B C * D E
^	+ (* (/ ^	A B C * D E
F	+ (* (/ ^	A B C * D E F
)	+ (*	A B C * D E F ^ /
*	+ (*	A B C * D E F ^ / *
G	+ (*	A B C * D E F ^ / * G
)	+	A B C * D E F ^ / * G *
*	+ *	A B C * D E F ^ / * G *
H	+ *	A B C * D E F ^ / * G * H
End	Empty	A B C * D E F ^ / * G * H * +

The last row is the answer. It performs E^F before division and both products before the final addition.

Q4: A linked stack, a BST, and traversal reconstruction

Full question Q4(a) [10 marks]. What is doubly linked list? Why it is used? Write a function/ algorithm to implement stack using linked list with following cases:

i. When stack is empty
ii. When stack is full

Q4(a): doubly linked lists and a linked stack

A doubly linked list stores prev/data/next, supporting movement both ways and efficient removal of a known node. It is useful for browser back/forward histories and editable sequences. A stack needs only one end, so a singly linked list already suffices:

Algorithm: PUSH using a linked stack

Step 0: Start.
Step 1: Allocate a node holding x. If allocation fails, report overflow and go to Step N.
Step 2: Set the new node next link to the old top, then make top point to the new node.
Step N: End.

Algorithm: POP using a linked stack

Step 0: Start.
Step 1: If top is null, report underflow and go to Step N.
Step 2: Save the top data, move top to the next node, release the old node, and report the saved value.
Step N: End.

Keep the updated top pointer; the following operations use that same pointer. Empty push sets the new node's next to null; empty pop underflows. A linked stack is full only on allocation failure unless explicitly bounded. Both operations take $\Theta(1)$ time. With doubly linked nodes, also set the old top's prev to the new node on push, and the new top's prev to null on pop.

Full question Q4(b) [10 marks].

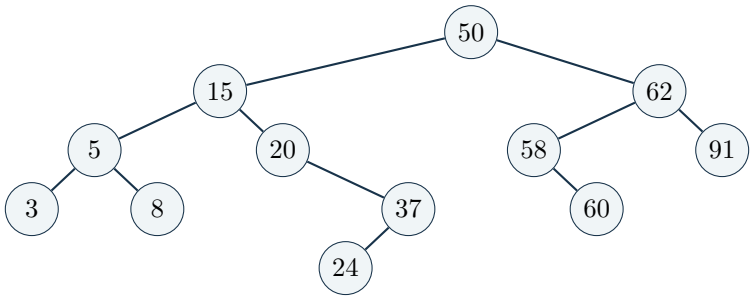
i. Construct a Binary Search Tree (BST) for the following sequence of numbers and Find the number of nodes in the left subtree and right subtree of the root.
50, 15, 62, 5, 20, 58, 91, 3, 8, 37, 60, 24

ii. Write an algorithm of postorder traversal in binary tree. Construct the binary tree whose traversals are given below.
Inorder: {4, 8, 2, 5, 1, 6, 3, 7},
Postorder: {8, 4, 5, 2, 6, 7, 3, 1}

Q4(b)(i): BST and subtree sizes

Insert $50, 15, 62, 5, 20, 58, 91, 3, 8, 37, 60, 24$ using page 5. New attachments: 15=L(50), 62=R(50), 5=L(15), 20=R(15), 58=L(62), 91=R(62), 3=L(5), 8=R(5), 37=R(20), 60=R(58), 24=L(37).

Key	Every comparison from the root	Attachment
50	Empty tree	Root
15	$15 < 50$	Left of 50
62	$62 > 50$	Right of 50
5	$5 < 50;\ 5 < 15$	Left of 15
20	$20 < 50;\ 20 > 15$	Right of 15
58	$58 > 50;\ 58 < 62$	Left of 62
91	$91 > 50;\ 91 > 62$	Right of 62
3	$3 < 50;\ 3 < 15;\ 3 < 5$	Left of 5
8	$8 < 50;\ 8 < 15;\ 8 > 5$	Right of 5
37	$37 < 50;\ 37 > 15;\ 37 > 20$	Right of 20
60	$60 > 50;\ 60 < 62;\ 60 > 58$	Right of 58
24	$24 < 50;\ 24 > 15;\ 24 > 20;\ 24 < 37$	Left of 37



Left subtree: 7 nodes $\{15, 5, 3, 8, 20, 37, 24\}$; **right subtree: 4 nodes** $\{62, 58, 60, 91\}$. Including root: $7 + 1 + 4 = 12$ nodes. Inorder check: $3, 5, 8, 15, 20, 24, 37, 50, 58, 60, 62, 91$.

Construction method: numbered recipe on page 25; staged walkthrough on page 25.

Q4(b)(ii): Reconstruct a binary tree from two traversals

This is a general binary tree, so do not sort the values or apply BST insertion. **Inorder** visits left subtree, root, right subtree. **Postorder** visits left subtree, right subtree, root. Therefore the last postorder value tells us the root; finding that root in inorder tells us exactly which labels belong on each side. The number of labels on the inorder left tells us how many postorder labels to allocate to the left subtree.

Algorithm: Postorder traversal of a binary tree

Step 0: Start.
Step 1: If the supplied subtree is empty, go to Step N.
Step 2: Traverse the left subtree; traverse the right subtree; visit the root. For each recursive traversal, apply this same algorithm to that child subtree.
Step N: End.

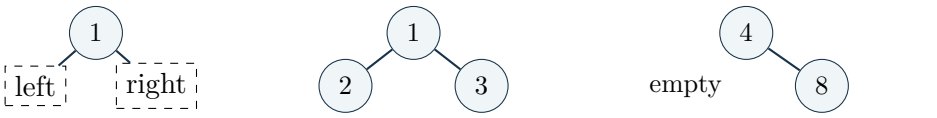
Algorithm: Build a subtree from its inorder and postorder lists

Step 0: Start.
Step 1: If both lists are empty, give an empty subtree to the caller and go to Step N. If their lengths or label sets disagree, report invalid traversals and go to Step N.
Step 2: Take the last label in P as root r and create its node.
Step 3: Find r in I. Let I_L be the labels before it and I_R the labels after it. Let k be the length of I_L.
Step 4: Remove the final root from P. Its first k labels form P_L; its remaining labels form P_R. Preserve their order.
Step 5: Apply this algorithm to (I_L, P_L); attach the resulting root as the left child of r.
Step 6: Apply this algorithm to (I_R, P_R); attach the result as the right child of r.
Step 7: Give the completed node r to the caller. The outermost call produces the whole tree root.
Step N: End.

The complete recursive partition. The empty left subtree of node 4 is essential: 8 lies after 4 in inorder, so it must be 4's right child.

Subproblem	Inorder	Postorder	Root
Whole	$4, 8, 2, 5, 1, 6, 3, 7$	$8, 4, 5, 2, 6, 7, 3, 1$	1
Left of 1	$4, 8, 2, 5$	$8, 4, 5, 2$	2
Left of 2	$4, 8$	$8, 4$	4
Left of 4	Empty	Empty	None
Right of 4	8	8	8
Right of 2	5	5	5
Right of 1	$6, 3, 7$	$6, 7, 3$	3
Left of 3	6	6	6
Right of 3	7	7	7

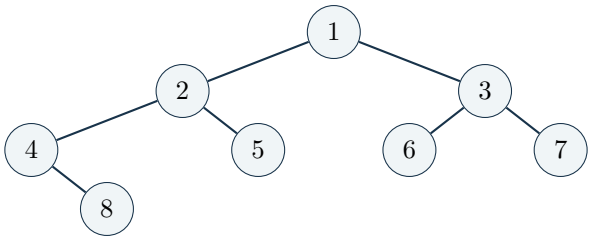
1. Identify root **2. Locate subtree roots** **3. Resolve the $[4, 8]$ pair**



Repeated linear searches for roots in inorder cost $O(n^2)$ in the worst case. A precomputed value-to-index map with index ranges gives $O(n)$ construction time, with $O(n)$ map storage and $O(h+1)$ recursive stack space.

Q4(b)(ii): inorder $4, 8, 2, 5, 1, 6, 3, 7$; postorder $8, 4, 5, 2, 6, 7, 3, 1$

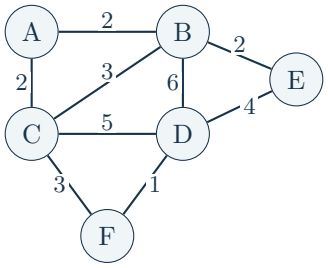
Postorder traversal recursively processes left, then right, then root (full numbered algorithm: page 24). To reconstruct, take the final postorder item 1 as root. Inorder splits as $[4, 8, 2, 5] \mid 1 \mid [6, 3, 7]$. Left postorder $[8, 4, 5, 2]$ gives root 2; right postorder $[6, 7, 3]$ gives root 3. Under 2, inorder $[4, 8]$ gives root 4 with *right* child 8, and the right child of 2 is 5. Under 3, children are 6 and 7.



Preorder is $1, 2, 4, 8, 5, 3, 6, 7$. Re-reading inorder and postorder reproduces both supplied sequences.

Q5: Prim from A and another BST

Full question Q5(a) [10 marks]. Explain steps in Prim's Algorithm. Find the minimum spanning tree and the cost of the graph given below using Prim's Algorithm. Consider vertex A as starting vertex.



Q5(a): minimum spanning tree

Use the same Prim rule as Paper A Q6(a): one endpoint inside, the other outside. The minimum edge anywhere in the graph is FD:1, but from A it is initially ineligible. A tie may be resolved in more than one way; the stated alphabetical rule makes this trace reproducible.

Undirected edges are AB:2, AC:2, BC:3, BD:6, BE:2, CD:5, CF:3, DE:4, DF:1. Prim starts at A and chooses only edges crossing from its included set to an outside vertex. For tied weights, order undirected edge labels alphabetically.

Step	Included before	Choose	Explanation / running cost
1	$\{A\}$	AB:2	Tie with AC; choose AB; cost 2
2	$\{A, B\}$	AC:2	Tie with BE; choose AC; cost 4
3	$\{A, B, C\}$	BE:2	Cheapest crossing edge; cost 6
4	$\{A, B, C, E\}$	CF:3	Beats ED:4, CD:5, BD:6; cost 9
5	$\{A, B, C, E, F\}$	FD:1	Becomes eligible after F joins; cost 10

MST edges $\{AB, AC, BE, CF, FD\}$**; cost** $2 + 2 + 2 + 3 + 1 = \boxed{10}$**.** Six vertices and five connected, acyclic edges confirm a spanning tree. FD is the graph's cheapest edge, but cannot be chosen before either F or D is in Prim's tree. The conceptual algorithm and its implementations' costs are in page 13.

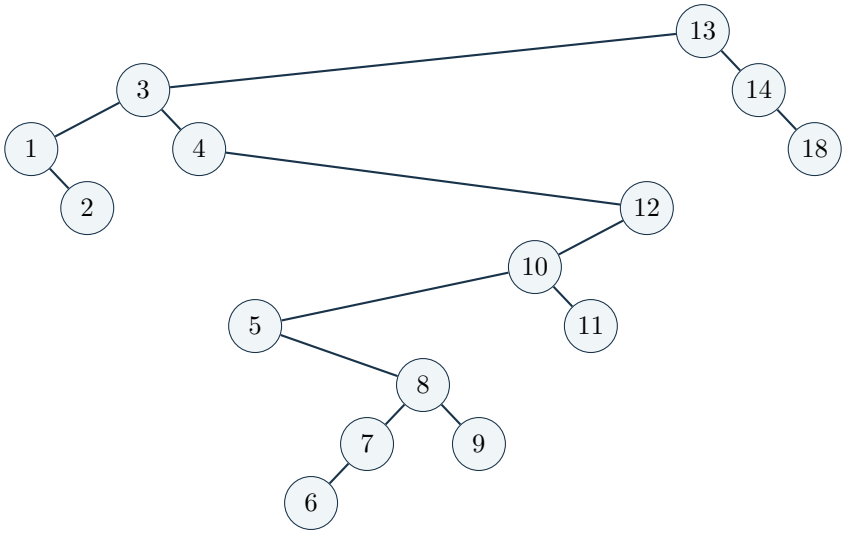
Q5(b): Insert fifteen keys and read the BST

Full question Q5(b) [10 marks]. Write an algorithm/Function to insert a node in binary search tree (BST). Construct a Binary Search tree using the given numbers.
13, 3, 4, 12, 14, 10, 5, 1, 8, 2, 7, 9, 11, 6, 18.

Use `root=BST_INSERT(root,key)` from page 5: compare, follow left/right, allocate at null, and return updated links. Input:

$$13, 3, 4, 12, 14, 10, 5, 1, 8, 2, 7, 9, 11, 6, 18.$$

Key	Every comparison from the root	Attachment
13	Empty tree	Root
3	$3 < 13$	Left of 13
4	$4 < 13;\ 4 > 3$	Right of 3
12	$12 < 13;\ 12 > 3;\ 12 > 4$	Right of 4
14	$14 > 13$	Right of 13
10	$10 < 13;\ 10 > 3;\ 10 > 4;\ 10 < 12$	Left of 12
5	$5 < 13;\ 5 > 3;\ 5 > 4;\ 5 < 12;\ 5 < 10$	Left of 10
1	$1 < 13;\ 1 < 3$	Left of 3
8	$8 < 13;\ 8 > 3;\ 8 > 4;\ 8 < 12;\ 8 < 10;\ 8 > 5$	Right of 5
2	$2 < 13;\ 2 < 3;\ 2 > 1$	Right of 1
7	$7 < 13;\ 7 > 3;\ 7 > 4;\ 7 < 12;\ 7 < 10;\ 7 > 5;\ 7 < 8$	Left of 8
9	$9 < 13;\ 9 > 3;\ 9 > 4;\ 9 < 12;\ 9 < 10;\ 9 > 5;\ 9 > 8$	Right of 8
11	$11 < 13;\ 11 > 3;\ 11 > 4;\ 11 < 12;\ 11 > 10$	Right of 10
6	$6 < 13;\ 6 > 3;\ 6 > 4;\ 6 < 12;\ 6 < 10;\ 6 > 5;\ 6 < 8;\ 6 < 7$	Left of 7
18	$18 > 13;\ 18 > 14$	Right of 14



For 6, compare against $13, 3, 4, 12, 10, 5, 8, 7$, then attach left of 7. This long path illustrates how an ordinary BST can be unbalanced. Inorder check: $1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 18$. Each insertion costs $O(h + 1)$; worst total construction cost is $O(n^2)$.

Q6: Linked-list insertion/deletion and merge sort

Full question Q6(a) [10 marks]. What is Linked list? Write a simple function / algorithm for

i) Inserting a new node after a given value in Singly Linked List
ii) Deleting a node from the beginning in the Singly linked list

Q6(a): define and operate on a singly linked list

A linked list is a sequence of separately allocated nodes connected by pointers. The first is reached through head; the last points to null. To insert after a given *value*, first locate it:

Algorithm: Insert after the first matching list value

Step 0: Start.
Step 1: Follow next links from head until the required value is found or the list ends.
Step 2: If absent, report not found and go to Step N.
Step 3: Apply the supplied-node insertion algorithm in page 8 to that node; report the resulting status.
Step N: End.

Algorithm: Remove the first node of a singly linked list

Step 0: Start.
Step 1: If head is null, report underflow and go to Step N.
Step 2: Keep the old head in q and save its data. Move head to the next node.
Step 3: Release q and report its saved data and the updated head.
Step N: End.

From $5 \to 10 \to 20 \to null$, insert 15 after 10 to get $5 \to 10 \to 15 \to 20 \to null$; deleting the beginning then returns 5 and leaves $10 \to 15 \to 20 \to null$. An absent insertion key leaves the list unchanged. Deleting the sole node makes head null.

Cost. Insertion by value is $O(n)$ including search; deletion at the head is $\Theta(1)$. Both use $\Theta(1)$ auxiliary space.

Q6(b): Merge-sort every run and comparison

Full question Q6(b) [10 marks]. Write an algorithm of merge sort with its time complexity. Sort the given element using merge sort and represent all the steps. 39, 9, 81, 45, 90, 27, 72, 18.

Q6(b): merge-sort $39, 9, 81, 45, 90, 27, 72, 18$

Call `MERGE_SORT(A,0,7)`; full algorithm and merge helper are in page 11. Split:

$$[39, 9, 81, 45] \mid [90, 27, 72, 18]$$
$$[39, 9] \mid [81, 45] \mid [90, 27] \mid [72, 18]$$
$$[39] \mid [9] \mid [81] \mid [45] \mid [90] \mid [27] \mid [72] \mid [18].$$

Merge singletons:

$$[9, 39] \mid [45, 81] \mid [27, 90] \mid [18, 72].$$

Merge pairs:

$$[9, 39, 45, 81] \mid [18, 27, 72, 90].$$

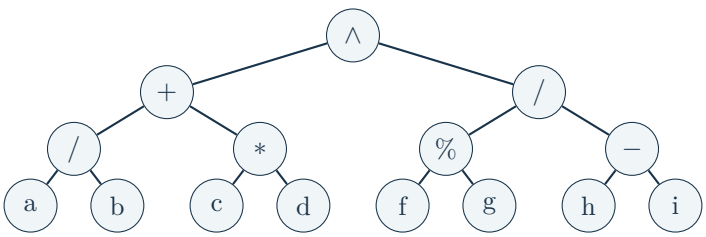
The left pair merge takes 9,39 then copies 45,81. The right pair merge compares 27/18 (take 18), 27/72 (take 27), 90/72 (take 72), then copies 90.

Final comparison	Take	Final comparison	Take
9 vs 18	9	45 vs 72	45
39 vs 18	18	81 vs 72	72
39 vs 27	27	81 vs 90	81
39 vs 72	39	Left exhausted	Copy 90

Result: $[9, 18, 27, 39, 45, 72, 81, 90]$**.** Time is $\Theta(n \log n)$ in all standard cases, auxiliary space $O(n)$; it is stable when ties are taken from the left run first.

Q7: Expression tree, probing, circular queue, sorting, DFS

Full question Q7(a) [5 marks]. Write the expression for the given binary tree. What is linear probing in hashing? Insert the elements 50, 70, 76, 93 using linear probing considering the mod 5.



Q7(a): read the expression tree

An expression tree has operators at internal nodes and operands at leaves. Read each subtree as one complete operand of its parent. Do not flatten a denominator into a sum or omit parentheses: the tree supplies the grouping even when normal precedence would suggest another grouping.

Its root is exponentiation. The left subtree is a sum of a/b and $c * d$; the right subtree divides f mod g by $h - i$. Thus the fully grouped expression is

$$\boxed{((a/b) + (c * d))^{((f \bmod g)/(h-i))}}.$$

Prefix: `^ + / a b * c d / % f g - h i`.

Postfix: `a b / c d * + f g % h i - / ^`.

`%` is the remainder operator; `^` denotes exponentiation in this paper. Each operator combines precisely the two child subexpressions, preserving their left/right order.

Q7(a), second task: hashing by linear probing

A hash function selects a home slot. With size 5, $h(k) = k$ mod 5 means the remainder after division by 5: $76 = 15(5) + 1$, so 76 starts at index 1. On collision, linear probing tries the next index, wrapping from 4 back to 0. It checks actual keys rather than assuming equal hash values mean equal records.

For linear probing, compute $h(k) = k$ mod 5, then try $(h(k) + i)$ mod 5 until empty:

Key	Hash	Probe indices	Store at
50	0	0	0
70	0	0 occupied, 1	1
76	1	1 occupied, 2	2
93	3	3	3

Index	Value
0	50
1	70
2	76
3	93
4	—

The load factor is 4/5. A collision means continue probing, not replace the existing key. General worst-case probing costs $O(m)$ for a table of size m.

Q7(b)–(d): Circular queues, sorting, and DFS

Full question Q7(b) [5 marks]. What do you understand by the concept of circular queue? How is it better than a linear queue?

Q7(b): circular queue versus linear queue

A circular queue wraps indices modulo capacity, reusing cells released at the front. A simple linear queue cannot reuse such cells while nonempty without shifting or resetting. For capacity 4, enqueue 1,2,3,4; dequeue 1,2; enqueue 5,6. The circular queue now contains $[3, 4, 5, 6]$ despite physical layout $[5, 6, 3, 4]$. Both insertion and removal are $O(1)$; circularity improves capacity reuse. Count-based empty/full conditions are $count = 0$ and $count = C$. See page 14.

Full question Q7(c) [5 marks]. Compare selection sort and insertion sort algorithms.

Q7(c): selection versus insertion sort

Selection scans the whole unsorted suffix for a minimum, then swaps it into position. It always makes $\Theta(n^2)$ comparisons and at most $n - 1$ swaps, uses $\Theta(1)$ extra space, and is generally unstable. Insertion shifts larger prefix values to insert the next key; best time is $\Theta(n)$, average/worst $\Theta(n^2)$, extra space $\Theta(1)$, and it is stable with a strict $>$ shift condition. It benefits from nearly sorted data. Numbered algorithms and pass-by-pass example: page 4.

Full question Q7(d) [5 marks]. Write an algorithm to explain Depth First Search with suitable example.

Q7(d): DFS algorithm and example

Algorithm: Depth-first search from a source vertex

Step 0: Start.
Step 1: Mark every vertex unvisited. Apply the following visit procedure to the source.
Step 2: For the current vertex u, mark it visited and record it.
Step 3: Examine its neighbours in the stated order. For each still-unvisited neighbour, apply the visit procedure of Steps 2–3 to that neighbour before examining the next one.
Step 4: When a call has no unvisited neighbour left, return to its caller. When the source call finishes, report the recorded traversal.
Step N: End.

Initialise all visited flags to false, then call DFS(A). For A–B, A–C, B–D, C–E, visit A, recurse B, recurse D, backtrack D to B to A, recurse C, recurse E. Order: **A B D C E**. Lists give $O(V + E)$ time and $O(V)$ auxiliary stack/visited space; repeated starts cover disconnected components.

3 Paper C — Final examination, 4 December 2023

Question 1(a)

Q1: Answer briefly. [20 marks in total]

Full question Q1(a) [5 marks]. Compare and contrast the structural differences between linear data structures (e.g., arrays and linked lists) and hierarchical data structures (e.g., trees). Identify the most suitable data structure based on a given scenario:

You are building a simple program to keep track of a to-do list. Each item on the list needs to be easily accessible for viewing, modification, and deletion. Which data structure would you choose and why?

Q1(a): linear versus hierarchical structures; a to-do list

A linear structure organises items in a single sequence. Arrays use contiguous indexed slots; linked lists use next links between nodes. A hierarchical structure organises parent–child branches: in a tree, a node may have several descendants at different depths. Trees suit nested folders or organisational hierarchies; a simple flat to-do list has no necessary hierarchy.

For a small to-do list whose items should be directly accessible for viewing and editing, choose a **dynamic array/list**. Access and replacement by index are $O(1)$; deletion by index shifts later entries and is $O(n)$; append is amortised $O(1)$ under standard geometric resizing. This is simple and usually adequate for a short list. A singly or doubly linked list is also defensible if frequent deletion through *already held node references* is the dominant requirement. Finding that node by position or value still costs $O(n)$. The prompt does not specify operation frequencies, so explain the tradeoff rather than claim one structure is optimal for every workload.

Q1(b): Convert infix to Polish notation with a stack

Full question Q1(b) [5 marks]. Given a mathematical expression in infix notation, demonstrate the step-by-step process of converting it into polish notation. Use a stack to show how operands and operators are processed.

`(A+B)+C-(D-E)^F`

Polish notation means *prefix*, whereas reverse Polish means postfix. An operator precedes its two operands in prefix: $x - y$ becomes `- x y`. A whole subexpression may be an operand: $(x + y) - z$ becomes `- + x y z`.

The outermost operation here is subtraction. Its left operand is $(A + B) + C$, which becomes `+ + A B C`. Its right operand is $(D - E)^F$, which becomes `^ - D E F`. Placing the outer subtraction first predicts the final answer before we verify it by a stack trace.

Algorithm: Infix to prefix by a right-to-left operator stack

Step 0: Start.
Step 1: Initialise an empty stack and an empty temporary output; scan tokens right to left.
Step 2: Append an operand to temporary output. Push a closing parenthesis `)` onto the stack.
Step 3: At an opening parenthesis `(`, pop operators to temporary output until the matching `)` is found, then discard that parenthesis. If none exists, report invalid input and go to Step N.
Step 4: At an operator, pop operators of greater precedence, and equal precedence only if the incoming operator is right associative. Stop at a parenthesis; then push the incoming operator.
Step 5: Repeat Steps 2–4 for the remaining tokens. Pop all remaining operators; a remaining parenthesis means invalid input.
Step 6: Reverse the temporary output *token sequence* and report it as prefix.
Step N: End.

The equal-precedence rule differs from the left-to-right postfix rule because the scan direction is reversed. Reverse tokens, not the characters inside a multidigit operand.

Grouping is $((A + B) + C) - ((D - E)^F)$. Polish means prefix. One stack method scans right to left, appending to a temporary output; reverse that output at the end. Push `)`; at `(`, pop until `)`. For an operator, pop higher precedence, or equal precedence if the incoming operator is *right associative*; then push it. This equal-precedence rule is reversed from ordinary left-to-right postfix conversion.

Token (right to left)	Stack after token	Temporary output
F	Empty	F
^	^	F
)	)	F
E	)	F E
-	) -	F E
D	) -	F E D
(	^	F E D -
-	-	F E D - ^
C	-	F E D - Ĉ
+	- +	F E D - Ĉ
)	- +)	F E D - Ĉ
B	- +)	F E D - Ĉ B
+	- +) +	F E D - Ĉ B
A	- +) +	F E D - Ĉ B A
(	- +	F E D - Ĉ B A +
End: pop all	Empty	F E D - Ĉ B A + + -

Reverse the temporary token sequence:

$$\boxed{\texttt{- + + A B C ^ - D E F}}.$$

Check structurally: outer `-` combines `+ + A B C` with `^ - D E F`. The first is $(A + B) + C$ and the second $(D - E)^F$. Stack conversion takes $O(n)$ time and space.

Q1(c): an alphabetical linked list; Q1(d): DFS from G

Full question Q1(c) [5 marks]. A classroom contains 10 benches of which 7 are occupied as shown below. Draw the memory representation of linked list for storing the student's names in alphabetical order.

Draw the memory representation of linked list if student allocated bench number 3 and 7 leaves classroom in linked list created in I

Draw the memory representation of linked list after allocating Roma bench number 8 in above linked list

Bench Number	1	2	3	4	5	6	7	8	9	10
Student name	Kavita		Diksha	Adil	Adnan		Lara		Sahil	Firoz

Q1(c): initial memory representation

The occupied benches are 1:Kavita, 3:Diksha, 4:Adil, 5:Adnan, 7:Lara, 9:Sahil, 10:Firoz. Use symbolic addresses B1,B3,..., retaining each student's bench number. Alphabetical list:

Address	Bench	Name	next
B4	4	Adil	B5
B5	5	Adnan	B3
B3	3	Diksha	B10
B10	10	Firoz	B1
B1	1	Kavita	B7
B7	7	Lara	B9
B9	9	Sahil	null

Head is B4. When benches 3 and 7 leave, set B5.next=B10 and B1.next=B9, and free B3/B7. Chain becomes Adil → Adnan → Firoz → Kavita → Sahil → *null*. Then allocate B8=(8,Roma), set B8.next=B9

and B1.next=B8:

$$head \to Adil_4 \to Adnan_5 \to Firoz_{10} \to Kavita_1 \to Roma_8 \to Sahil_9 \to null.$$

Only links determine order. No node is needed for an empty bench. These are sequential updates after the departures.

Q1(d): DFS, backtracking, and the faint graph

Full question Q1(d) [5 marks]. Define degree of a node in the context of a graph. Identify DFS sequence for below graph. Consider source node as G. Show each step.

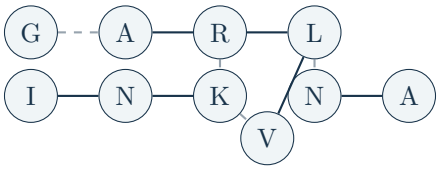
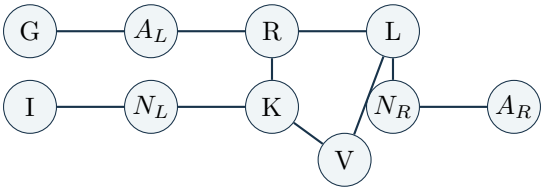


Figure transcription: dashed pale connections are faint in the supplied scan; the solution states its interpretation.

Q1(d): degree and a careful reading of the faint graph

Undirected degree is the number of incident edges. Directed graphs have separate indegree/outdegree. The scanned undirected diagram repeats A and N. Treat distinct circles as distinct vertices: A_L beside G, A_R at the far right; N_L beside I, N_R below L. Faint lines are interpreted as G–A_L, R–K, K–V, and L–N_R, in addition to the clearer lines below. The answer is conditional on that reading of the scan.



Adjacency: G:A_L; A_L:G,R; R:A_L,K,L; K:N_L,R,V; N_L:I,K; I:N_L; V:K,L; L:N_R,R,V; N_R:A_R,L; A_R:N_R. Alphabetical DFS: G $\to A_L \to R \to K \to N_L \to I$; backtrack I,$N_L$ to K; next V $\to L \to N_R \to A_R$; then backtrack fully, skipping visited R and V. Thus

$$\boxed{G, A_L, R, K, N_L, I, V, L, N_R, A_R}.$$

Step	Vertex first visited	Next action / return
0	G	Mark G, choose A_L
1	A_L	Skip visited G, choose R
2	R	Skip A_L; choose K before L
3	K	Choose N_L before V
4	N_L	Choose I
5	I	No new neighbour: return to N_L, then K
6	V	K continues with V, which chooses L
7	L	Choose N_R; R and V are already visited
8	N_R	Choose A_R
9	A_R	No new neighbour; return through unfinished calls

For example $\deg(R) = 3$, $\deg(K) = 3$, $\deg(G) = 1$. Repeated printed labels do not merge separate circles. If G–A_L is absent in a clearer original, G is isolated and source DFS would contain only G.

Q2: Array insertion, product complexity, and circular queue

Full question Q2(a) [10 marks]. Write a pseudo code function to insert an element in an integer array at a specified position by shifting elements to the right.
Determine the time complexity in Big-O notation of the following function.

```
int multiply(int A[], int N)
{ int s=1;
  for (int i=0; i<N; i++)
    s = s * A[i];
  return s; }
```

Q2(a): insertion at a specified position

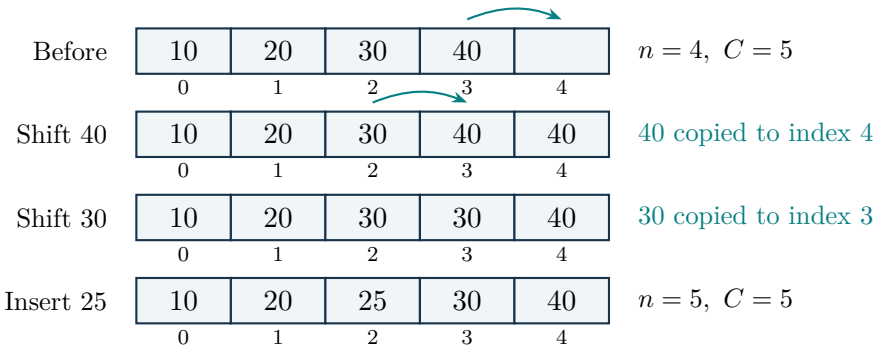
Array positions hold one value each. Inserting in the middle needs a new vacant cell, so every existing item at or after the target position moves one slot right. Shift from the *rightmost* item backwards: shifting left-to-right would overwrite a value before it is copied. The array length counts occupied cells; capacity counts all available cells. Insertion changes length but does not automatically create capacity.

Use length n, capacity C, and zero-based position p. Positions 0 through n are valid, with $p = n$ meaning append.

Algorithm: Insert x at zero-based position p

Step 0: Start.
Step 1: Read array A, length n, capacity C, position p, and new value x.
Step 2: If $n = C$, report overflow and go to Step N. If $p < 0$ or $p > n$, report an invalid position and go to Step N.
Step 3: Set $i = n - 1$, the last occupied index.
Step 4: While $i \geq p$, copy $A[i]$ into $A[i+1]$, then decrease i by 1.
Step 5: Write x into $A[p]$ and increase n by 1.
Step 6: Report the updated array and length.
Step N: End.

Example: insert 25 at index 2 in $[10, 20, 30, 40]$. Copy 40 from 3 to 4, then 30 from 2 to 3, then write 25 at 2. Result $[10, 20, 25, 30, 40]$. Backward shifting preserves old values. There are $n - p$ shifts; time is $\Theta(n - p + 1)$, worst $O(n)$, and extra space $\Theta(1)$.



The supplied `multiply(A,N)` starts $s = 1$, repeats $s = s * A[i]$ for $i = 0, \ldots, N-1$, and returns s. Its loop has N multiplications, N increments, and $N + 1$ condition checks. Therefore $T(N) = aN + b = \boxed{\Theta(N)}$, and in particular $O(N)$. Auxiliary space is $\Theta(1)$. This uses fixed-size unit-cost arithmetic; exact arbitrarily large products have a growing arithmetic cost. For $N = 0$, the result is the empty product 1.

Q2(b): Circular queue insertion, removal, and display

Full question Q2(b) [10 marks]. Write an algorithm to perform en-queue, de-queue and display operations on a circular queue using arrays. Check for full and empty queue conditions wherever required.

Use the count convention: $front = rear = count = 0$ initially and rear is the next free slot.

Algorithm: ENQUEUE in a circular array queue

Step 0: Start.
Step 1: If $count = C$, report overflow and go to Step N.
Step 2: Store x at index rear; replace rear by $(rear + 1) \bmod C$.
Step 3: Increase count by 1 and report success.
Step N: End.

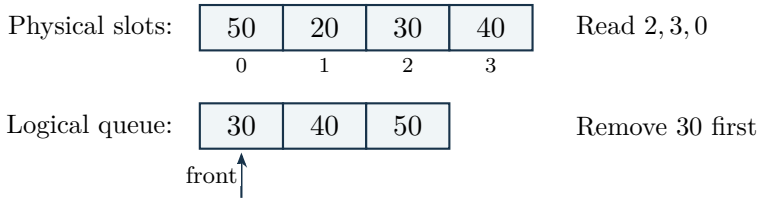
Algorithm: DEQUEUE in a circular array queue

Step 0: Start.
Step 1: If $count = 0$, report underflow and go to Step N.
Step 2: Save the value at index front; replace front by $(front + 1) \bmod C$.
Step 3: Decrease count by 1 and report the saved value.
Step N: End.

Algorithm: DISPLAY a circular queue in service order

Step 0: Start.
Step 1: If $count = 0$, report empty and go to Step N.
Step 2: For $k = 0, 1, \ldots, count - 1$, display $Q[(front + k) \bmod C]$.
Step N: End.

Capacity 4 example: enqueue 10,20,30; dequeue twice; enqueue 40,50. State is $front = 2, rear = 1, count = 3$ and physical cells contain $[50, \text{stale } 20, 30, 40]$. Display reads indices 2,3,0, giving $[30, 40, 50]$. The stale cell is not a current queue element.



The physical array is not the service order after wrapping. Display starts at front, advances one index modulo capacity, and stops after count items. It must not scan indices 0 through $C - 1$ indiscriminately, because some cells may contain stale values outside the current queue.

Cost. Enqueue/dequeue are $\Theta(1)$; display is $\Theta(count)$. Array storage is $\Theta(C)$. Count 0 means empty, count C means full, even if front equals rear in both cases.

Q3: Selection sort; a matrix and Dijkstra from D

Full question Q3(a) [10 marks]. Apply selection sort on an array given and show each pass for the sorting.

Index	0	1	2	3	4	5	6	7	8
Values	45	32	12	23	30	6	15	40	17

Q3(a): all selection-sort passes

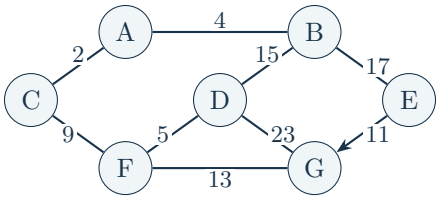
Input is $[45, 32, 12, 23, 30, 6, 15, 40, 17]$. At each pass, find the minimum of the entire unsorted suffix, then swap it into the next position. Algorithm: page 4.

Pass	Selected minimum / swap	Array after pass
0	—	$45, 32, 12, 23, 30, 6, 15, 40, 17$
1	6; swap indices 0,5	$6, 32, 12, 23, 30, 45, 15, 40, 17$
2	12; swap 1,2	$6, 12, 32, 23, 30, 45, 15, 40, 17$
3	15; swap 2,6	$6, 12, 15, 23, 30, 45, 32, 40, 17$
4	17; swap 3,8	$6, 12, 15, 17, 30, 45, 32, 40, 23$
5	23; swap 4,8	$6, 12, 15, 17, 23, 45, 32, 40, 30$
6	30; swap 5,8	$6, 12, 15, 17, 23, 30, 32, 40, 45$
7	32; already at 6	$6, 12, 15, 17, 23, 30, 32, 40, 45$
8	40; already at 7	$6, 12, 15, 17, 23, 30, 32, 40, 45$

There are $8 + 7 + \cdots + 1 = 36$ comparisons and 6 actual swaps here. Time is $\Theta(n^2)$ regardless of existing order; extra space $\Theta(1)$.

Q3(b): Directed adjacency matrix and Dijkstra from D

Full question Q3(b) [10 marks]. Represent below graph as an adjacency matrix. Apply Dijkstra's algorithm for the following graph taking D as initial node. Show each step.



Use the numbered Dijkstra procedure on page 21. In this graph E has arrows leaving it, but no route from D enters E. An edge can be followed only from its tail to its arrowhead. Treating the graph as undirected would change both reachability and the shortest paths.

The scan gives $B \to A : 4$, $A \to C : 2$, $D \to B : 15$, $D \to F : 5$, $D \to G : 23$, $F \to C : 9$, $F \to G : 13$, $E \to B : 17$, $E \to G : 11$. Rows are sources; order is A,B,C,D,E,F,G. A weighted adjacency matrix is

	A	B	C	D	E	F	G
A	0	∞	2	∞	∞	∞	∞
B	4	0	∞	∞	∞	∞	∞
C	∞	∞	0	∞	∞	∞	∞
D	∞	15	∞	0	∞	5	23
E	∞	17	∞	∞	0	∞	11
F	∞	∞	9	∞	∞	0	13
G	∞	∞	∞	∞	∞	∞	0

For a 0/1 adjacency matrix, replace every finite off-diagonal edge weight with 1, all missing-edge infinities with 0, and retain diagonal 0.

Finalised	A	B	C	D	E	F	G
Initial	∞	∞	∞	0	∞	∞	∞
D	∞	15	∞	0	∞	5	23
F	∞	15	14	0	∞	5	18
C	∞	15	14	0	∞	5	18
B	19	15	14	0	∞	5	18
G	19	15	14	0	∞	5	18
A	19	15	14	0	∞	5	18

Work through the choices. After D, the finite tentative distances are F:5, B:15, G:23. Select F, not B. F gives C:$5 + 9 = 14$ and improves G to $5 + 13 = 18$. Select C at 14; it has no outgoing edge. Select B at 15; it gives A:$15 + 4 = 19$. Select G at 18; it has no outgoing edge. Select A at 19; its proposed path to C costs $19 + 2 = 21$, worse than 14, so no update is made. E remains infinite, so stop rather than inventing a route to it.

F improves G to $5 + 13 = 18$ and gives C cost 14. B gives A cost $15 + 4 = 19$; A's route to C costs 21 and does not improve 14. Final paths: F: D–F (5); C: D–F–C (14); B: D–B (15); G: D–F–G (18); A: D–B–A (19). **E is unreachable**; its arrows point outward to B/G. Algorithm: page 21.

Q4: BST insertion, three traversals, reconstruction

Full question Q4(a) [10 marks]. Write an algorithm to perform insert operation in to binary search tree. Create a binary search tree by inserting the following elements in the given order.
60, 25, 72, 15, 30, 68, 101, 13, 18.

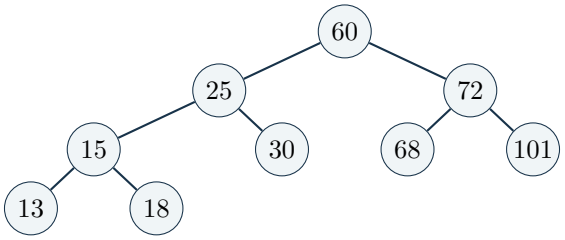
Q4(a): insert $60, 25, 72, 15, 30, 68, 101, 13, 18$

Algorithm: Insert a key into a distinct-key BST

Step 0: Start.
Step 1: If the supplied subtree is empty, allocate a leaf for the key, give it null children, and give its address to the caller; go to Step N. Report allocation failure if no node can be obtained.
Step 2: If the key is smaller than the root key, apply this algorithm to the left subtree and reconnect the resulting left link.
Step 3: If the key is larger, do the same for the right subtree. If equal, leave the tree unchanged under the stated duplicate policy.
Step 4: Give the updated subtree root to the caller; the outermost caller updates the tree root.
Step N: End.

New attachments: 60 is root; 25=L(60), 72=R(60), 15=L(25), 30=R(25), 68=L(72), 101=R(72), 13=L(15), 18=R(15). For 18, comparisons are $18 < 60$, $18 < 25$, $18 > 15$.

Key	Every comparison from the root	Attachment
60	Empty tree	Root
25	$25 < 60$	Left of 60
72	$72 > 60$	Right of 60
15	$15 < 60$; $15 < 25$	Left of 25
30	$30 < 60$; $30 > 25$	Right of 25
68	$68 > 60$; $68 < 72$	Left of 72
101	$101 > 60$; $101 > 72$	Right of 72
13	$13 < 60$; $13 < 25$; $13 < 15$	Left of 15
18	$18 < 60$; $18 < 25$; $18 > 15$	Right of 15



Inorder check: $13, 15, 18, 25, 30, 60, 68, 72, 101$. Preorder: $60, 25, 15, 13, 18, 30, 72, 68, 101$. Postorder: $13, 18, 15, 30, 25, 68, 101, 72, 60$. Insertion costs $O(h + 1)$ per key, with $O(n^2)$ worst-case total over n insertions.

Q4(b): Three traversals and the repeated reconstruction

Full question Q4(b) [10 marks]. Write an algorithm to perform in-order, post-order and pre-order traversal on binary tree.

Construct Binary Tree from given traversals

In-order sequence: 4, 8, 2, 5, 1, 6, 3, 7

Post-order sequence: 8, 4, 5, 2, 6, 7, 3, 1.

Q4(b): traversal algorithms

Algorithm: Preorder traversal of a binary tree

Step 0: Start.
Step 1: If the supplied subtree is empty, go to Step N.
Step 2: Visit the root; traverse its left subtree; traverse its right subtree. For each recursive traversal, apply this same algorithm to that child subtree.
Step N: End.

Algorithm: Inorder traversal of a binary tree

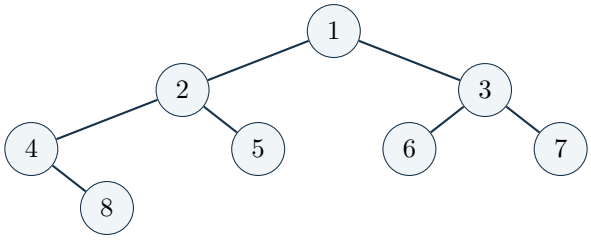
Step 0: Start.
Step 1: If the supplied subtree is empty, go to Step N.
Step 2: Traverse the left subtree; visit the root; traverse the right subtree. For each recursive traversal, apply this same algorithm to that child subtree.
Step N: End.

Algorithm: Postorder traversal of a binary tree

Step 0: Start.
Step 1: If the supplied subtree is empty, go to Step N.
Step 2: Traverse the left subtree; traverse the right subtree; visit the root. For each recursive traversal, apply this same algorithm to that child subtree.
Step N: End.

The separate numbered algorithms are also given in page 24. Each visits every node once: $\Theta(n)$ time and $O(h+1)$ call-stack space.

Use the numbered reconstruction recipe on page 25 and its staged walkthrough on page 25. Reconstruct the binary tree with inorder $[4, 8, 2, 5, 1, 6, 3, 7]$ and postorder $[8, 4, 5, 2, 6, 7, 3, 1]$. The final postorder value is root 1. Left inorder is $[4, 8, 2, 5]$, right inorder $[6, 3, 7]$; corresponding postorders are $[8, 4, 5, 2]$ and $[6, 7, 3]$. The left root is 2 with subtrees $[4, 8]$ and $[5]$; its left root 4 has right child 8. The right root 3 has children 6,7.

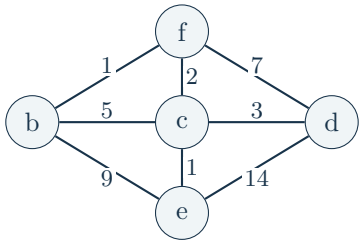


Verification: inorder is $4, 8, 2, 5, 1, 6, 3, 7$ and postorder is $8, 4, 5, 2, 6, 7, 3, 1$. This is the same reconstruction as Paper B Q4(b)(ii), so both papers share that worked method.

Q5: Kruskal, adjacency lists, and BFS reachability

Full question Q5(a) [10 marks]. Discuss two applications of minimum spanning tree. Represent below graph in adjacency list form.

Apply Kruskal' algorithm on below graph and identify the cost of connecting all the nodes. Show working at each step and final minimum spanning tree.



Q5(a): MST applications and weighted adjacency list

Prim grows one tree; **Kruskal** starts with every vertex separate and repeatedly joins different components using the next cheapest edge. A component is a set of vertices already connected by chosen edges. If an edge's endpoints are already in the same component, another path already joins them, so adding that edge creates a cycle.

Algorithm: Kruskal minimum spanning tree

Step 0: Start.
Step 1: Sort all edges by nondecreasing weight. Put each vertex in its own component; begin an empty selected-edge set and cost 0.
Step 2: Take the next edge. If no edge remains, go to Step 5.
Step 3: If its endpoints lie in different components, select the edge, merge the components, and add its weight to the cost. Otherwise discard it.
Step 4: If $V-1$ edges have been selected, go to Step 5; otherwise repeat Step 2.
Step 5: Report the selected edges and cost. Fewer than $V-1$ selected edges means a forest rather than a spanning tree.
Step N: End.

Applications include minimum-cost wiring/pipelines and single-linkage clustering. The graph's top vertex is printed f (lowercase); use vertices b,c,d,e,f. It has edges bf:1, fc:2, fd:7, bc:5, cd:3, be:9, ce:1, de:14. Its undirected weighted adjacency list is

b	$(f,1),(c,5),(e,9)$
c	$(f,2),(b,5),(d,3),(e,1)$
d	$(f,7),(c,3),(e,14)$
e	$(b,9),(c,1),(d,14)$
f	$(b,1),(c,2),(d,7)$

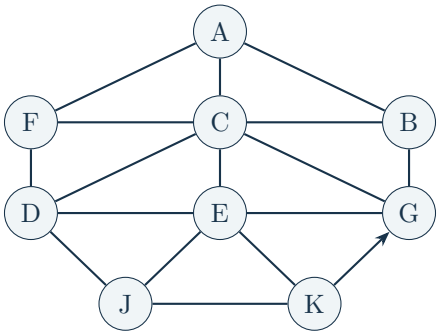
Kruskal sorts all edges, adding one only when it joins different components. Related explanation and method: page 13.

Order	Edge	Decision	Components after decision
1	bf:1	Accept	$\{b,f\},\{c\},\{d\},\{e\}$
2	ce:1	Accept	$\{b,f\},\{c,e\},\{d\}$
3	cf:2	Accept	$\{b,c,e,f\},\{d\}$
4	cd:3	Accept	$\{b,c,d,e,f\}$
5	bc:5	Stop / would reject	Already connected; cycle
6	df:7	Would reject	Cycle
7	be:9	Would reject	Cycle
8	de:14	Would reject	Cycle

MST edges $\{bf,ce,cf,cd\}$**, cost** $1+1+2+3=\boxed{7}$**.** It is the tree $b-f-c-d$ with e attached to c. Stop after 4 accepted edges for 5 vertices.

Q5(b): Breadth-first search and reachable vertices

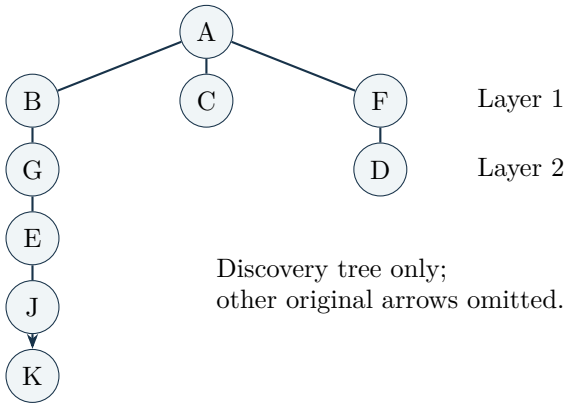
Full question Q5(b) [10 marks]. For the graph given below, apply BFS Algorithm to find all nodes reachable from node A. Show all the steps. Write an algorithm for the breadth first search traversal on graph.



Breadth-first search (BFS) finishes all vertices one edge away before those two edges away, then three, and so on. A FIFO queue preserves this discovery order. Mark a vertex *when it is enqueued*, so multiple arrows to the same vertex do not cause duplicate queue entries. Unlike Dijkstra, BFS minimises the number of edges, not arbitrary weighted costs.

Algorithm: Breadth-first search from a source vertex

Step 0: Start.
Step 1: Mark every vertex unvisited. Create an empty queue, mark the source visited, and enqueue it.
Step 2: If the queue is empty, report the traversal and go to Step N.
Step 3: Dequeue the front vertex u and record it.
Step 4: Examine outgoing neighbours in the stated order. For each unvisited neighbour v, mark it immediately, record u as its parent, and enqueue it. Go to Step 2.
Step N: End.



Q5(b): directed BFS from A

Read outgoing lists alphabetically:

A	B, C, F	B	C, G
C	F	D	C
E	C, D, J	F	D
G	C, E	J	D, K
K	E, G		

In particular, J points to D; that diagonal is not D to J. Initialise visited[A], enqueue A, then repeatedly dequeue and enqueue each unvisited outgoing neighbour. Full numbered algorithm is in page 10.

Step	Dequeue	Newly discovered	Queue afterwards
0	—	A	$[A]$
1	A	B,C,F	$[B, C, F]$
2	B	G	$[C, F, G]$
3	C	None (F already queued)	$[F, G]$
4	F	D	$[G, D]$
5	G	E	$[D, E]$
6	D	None (C visited)	$[E]$
7	E	J	$[J]$
8	J	K	$[K]$
9	K	None	Empty

BFS order: A,B,C,F,G,D,E,J,K. All 9 vertices are reachable. BFS layers are $\{A\}$, $\{B, C, F\}$, $\{G, D\}$, $\{E\}$, $\{J\}$, $\{K\}$. With adjacency lists, time is $O(V + E)$ and extra space $O(V)$.

Q6: Linked-list types, end insertion, and binary search

Full question Q6(a) [10 marks]. Explain different types of linked list. Write an algorithm for adding a new node at the end of a singly linked list and removing the first node from the singly linked list.

Q6(a): linked-list types

A singly linked list has next links and forward traversal. A doubly linked list also stores prev, supporting backward traversal and easy deletion of a known node at extra pointer cost. A circular singly linked list connects last.next to head; a circular doubly linked list connects last and first in both directions. In a nonempty circular list, traverse until returning to the start, rather than waiting for null.

For the requested operations use an ordinary *singly* linked list:

Algorithm: Append a node to a singly linked list

Step 0: Start.
Step 1: Allocate a node q holding x and give it a null next link. If allocation fails, report failure and go to Step N.
Step 2: If head is null, make head point to q and go to Step 4.
Step 3: Otherwise follow next links from head to the last node and set its next link to q.
Step 4: Report success and the updated head.
Step N: End.

Algorithm: Remove the first node of a singly linked list

Step 0: Start.
Step 1: If head is null, report underflow and go to Step N.
Step 2: Keep the old head in q and save its data. Move head to the next node.
Step 3: Release q and report its saved data and the updated head.
Step N: End.

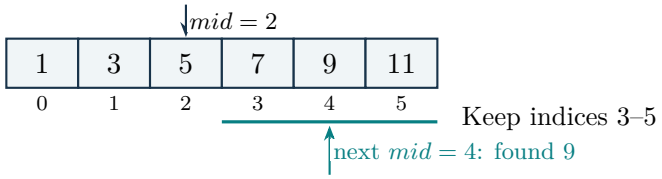
Example: start $10 \to 20 \to \mathit{null}$; append 30 to get $10 \to 20 \to 30 \to \mathit{null}$; remove-first returns 10 and leaves $20 \to 30 \to \mathit{null}$. On an empty list append sets head to the new node; on a one-node list remove-first makes head null. The caller must store the returned head.

Cost. Append is $O(n)$ without a tail pointer, $O(1)$ with a maintained tail. Head removal and auxiliary working space are $O(1)$.

Q6(b): Binary search and shrinking intervals

Full question Q6(b) [10 marks]. Write an algorithm for binary search. Perform binary search step-by-step on a given list for finding value 98. Show each step.

4, 21, 34, 40, 65, 76, 80, 92, 93, 97, 98



The precondition is a sorted array. If its middle value is too small, then every value at or before that middle index is also too small; discard that entire half. If the middle value is too large, discard the other half. Keep an inclusive interval $[lo, hi]$, meaning both endpoints are still candidates. Since mid has already been checked, use $mid + 1$ or $mid - 1$; retaining mid can prevent progress.

The sorted array is

$$[4, 21, 34, 40, 65, 76, 80, 92, 93, 97, 98].$$

Use `lo=0,hi=10`, inclusive, and `mid=lo+floor((hi-lo)/2)`.

Algorithm: Binary search in a sorted array

Step 0: Start.
Step 1: Read the sorted array A, length n, and target key. Set $lo = 0$ and $hi = n - 1$.
Step 2: If $lo > hi$, report not found and go to Step N.
Step 3: Compute $mid = lo + \lfloor (hi - lo)/2 \rfloor$. If $A[mid]$ equals the key, report mid and go to Step N.
Step 4: If $A[mid]$ is smaller than the key, set $lo = mid + 1$; otherwise set $hi = mid - 1$.
Step 5: Go to Step 2.
Step N: End.

Step	lo	hi	mid	A[mid]	Decision
1	0	10	5	76	$98 > 76$: set lo=6
2	6	10	8	93	$98 > 93$: set lo=9
3	9	10	9	97	$98 > 97$: set lo=10
4	10	10	10	98	Found; stop

98 is at index 10, the 11th position. Four comparisons are made. Worst time is $\Theta(\log n)$ and extra space is $\Theta(1)$ for the iterative method. The inclusive test `lo<=hi` is essential to inspect the final one-item interval.

Q7: Expression tree, Huffman code, and linear probing

Full question Q7(a) [5 marks]. Construct an expression tree for below expression. Identify prefix and postfix expression for the same. Show the working.

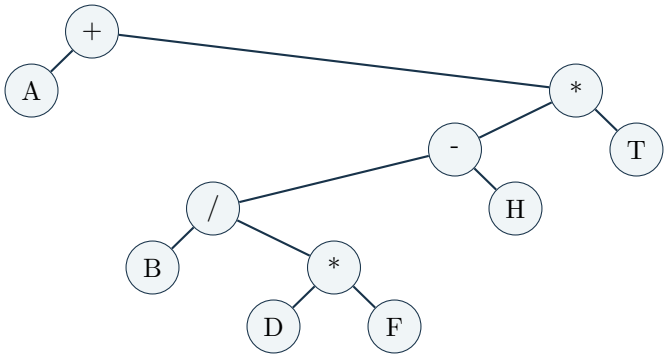
`A+(B/(D*F)-H)*T`

Q7(a): expression tree

Expression: $A + (B/(D * F) - H) * T$.

Build the expression from its innermost grouping outward. First $D * F$ is one subtree. Its parent / has left child B and right child that entire product. Subtract H from that quotient, multiply the result by T, then use that whole product as the right operand of the outer +. A goes on the left of +. An expression tree is not a BST: its operators impose no numerical left/right ordering rule.

Combine D,F with *, divide B by that subtree, subtract H, multiply by T, then add A. Each operator becomes the parent of its operands.



Read the two traversals carefully. For prefix, print + before its operands, then A, then the whole product subtree. Inside that product, print * before the subtraction and T; inside the subtraction, print - before the quotient and H. For postfix do the reverse placement: finish each left and right operand subtree before printing its operator. Thus D,F,* must appear before /, the quotient and H before -, and that subtraction and T before *.

Prefix: `+ A * - / B * D F H T`.

Postfix: `A B D F * / H - T * +`.

Preorder reads a root before its subtrees; postorder reads it afterwards. Grouped infix is $A + ((B/(D * F) - H) * T)$.

Q7(b)–(c): Huffman coding and linear probing

Full question Q7(b) [5 marks]. Construct the Huffman tree for the given symbols with their frequencies. Show each step. Identify the code for each letter.

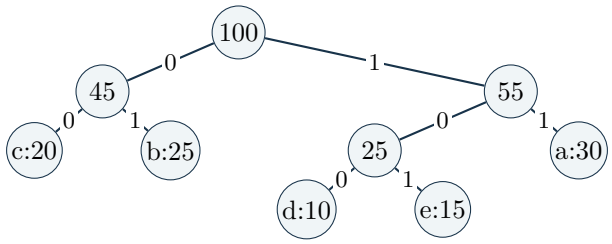
Ax= {a, b, c, d, e}

Px= {30, 25, 20, 10, 15}

Q7(b): Huffman tree

Frequencies: a:30, b:25, c:20, d:10, e:15. Merge the two current minima:

1. $d10 + e15 = 25$.
2. $c20 + b25 = 45$ (choose leaf b before the tied internal 25).
3. Internal $25 + a30 = 55$.
4. $45 + 55 = 100$.



Left=0, right=1 gives **a:11, b:01, c:00, d:100, e:101**.

The choice between b:25 and the internal (d,e):25 is a tie. Choosing b first here makes c and b siblings under 45. The internal 25 then joins a:30 under 55. Either tie policy is valid if both chosen trees have minimum available weights. To decode 100, start at 100, follow right to 55, left to 25, left to d; emit d and restart at the root. Code bits label edges; node weights are not code digits.

Weighted cost $= 30(2) + 25(2) + 20(2) + 10(3) + 15(3) = 225$ bits. Merge check: $25 + 45 + 55 + 100 = 225$; average 2.25 bits/symbol. Other tie choices can give different optimal codes with the same weighted cost.

Numerical interpretation. Each occurrence of a contributes two bits, and there are 30 such occurrences, giving 60 bits. Repeat this frequency-times-length calculation for every letter. With five

letters, a fixed-length code needs $\lceil \log_2 5 \rceil = 3$ bits per letter, or 300 data bits for 100 letters; the 225-bit Huffman payload saves 75 data bits, excluding its codebook.

Full question Q7(c) [10 marks]. Consider a hash table with size=10. Using linear probing insert, the keys 27, 33, 64, 43, 37, 29, 30 and 102 into the table. Discuss the disadvantages of linear probing.

Linear probing recipe. Divide the key by the table size and take the remainder as the first index. If occupied by another key, add one modulo the table size and try again. Stop at an empty cell, or after checking all cells if full. Never overwrite a colliding key.

Algorithm: Insert a new distinct key by linear probing

Step 0: Start.
Step 1: Set $j = h(key)$ and attempts=0. This insertion-only table uses EMPTY cells and has no deletions.
Step 2: If attempts equals table size m, report full and go to Step N.
Step 3: If cell j is EMPTY, store the key, report j, and go to Step N. If it already holds the key, report already present and go to Step N.
Step 4: Otherwise replace j by $(j + 1) \bmod m$ and increment attempts.
Step 5: Go to Step 2.
Step N: End.

If deletion is supported, use a DELETED marker (tombstone) and continue searches past it; turning it directly into EMPTY could break a collision chain.

Q7(c): size 10, $h(k) = k \bmod 10$, linear probing

Key	Hash	Probe sequence	Final index
27	7	7	7
33	3	3	3
64	4	4	4
43	3	3 occupied, 4 occupied, 5	5
37	7	7 occupied, 8	8
29	9	9	9
30	0	0	0
102	2	2	2

Index	0	1	2	3	4	5	6	7	8	9
Value	30	—	102	33	64	43	—	27	37	29

Disadvantages: primary clustering, longer probes at high load ($\alpha = 8/10$ here), tombstones needed for safe deletion, and possible resizing/rehashing. Worst-case operations are $O(m)$, despite expected $O(1)$ at suitable load and distribution. Probe modulo m and inspect at most m slots.

4 Repeated questions and examination patterns

What is being counted?

These are three distinct papers. Chronologically, C (December 2023) precedes B (February 2024), which precedes A (November 2025). The comparison below uses **printed top-level lettered subparts**: for example, Q1(b) counts once even if it has three internal tasks. This avoids assigning marks to internal tasks whose individual allocations are unclear. A has 16 such subparts, B has 18, and C has 17, for 51 in all.

An **exact data repeat** uses the same mathematical input for the same task. A **method repeat** reuses an algorithm or operation with changed data or wording. A **topic overlap** is broader still: two questions concern stacks, for example, but ask for different applications. These measures answer different questions and must not be added together.

The one confirmed identical worked problem

C Q4(b) and B Q4(b)(ii) both give

$$I = [4, 8, 2, 5, 1, 6, 3, 7], \qquad P = [8, 4, 5, 2, 6, 7, 3, 1]$$

and ask for the binary tree. The same root 1, left subtree rooted at 2, right subtree rooted at 3, and right child 8 under 4 solve both. This is an exact repeat of the reconstruction task, embedded in larger subparts that also request traversal algorithms or BST construction.

Conservative exact-repeat statement: one unique numerical task is confirmed as reused. Two of the 51 lettered subparts contain that task ($2/51 \approx 3.9\%$), but those whole subparts are *not* identical. Within the later B paper, one of its 18 lettered subparts contains a confirmed exact repeated task ($1/18 \approx 5.6\%$). These are occurrence statistics, not a percentage of marks. B allocates 10 marks to Q4(b) as a whole; the scan does not separately allocate the reconstruction marks, so an exact-repeat marks percentage cannot responsibly be calculated.

Concrete methods reused with changed inputs

Method / task	Paper C	Paper B	Paper A
BST insertion / construction	Q4(a)	Q4(b)(i), Q5(b)	Q1(c), Q5(b)
Traversal reconstruction	Q4(b)	Q4(b)(ii), identical	Q5(b), different traversals
Linked memory, ordered records	Q1(c), names	Q1(b), marks	List operations Q3(a), Q6(b)
Delete first list node	Q6(a)	Q6(a), same operation	Deletion by key Q6(b)
Circular-array queue	Q2(b)	Q7(b), comparison	Q7(a), operations
Selection sort	Q3(a), nine values	Q7(c), compare sorts	Q1(b), five values
DFS	Q1(d), from G	Q7(d), algorithm/example	Q4(a), from A
Dijkstra	Q3(b), from D	Q3(a), from 1	Not asked
MST construction	Q5(a), Kruskal	Q5(a), Prim from A	Q6(a), Prim from 5
Expression notation / tree	Q1(b), Q7(a)	Q2(a)(ii), Q3(b), Q7(a)	Q2(b), Q5(a)
Huffman coding	Q7(b), five symbols	Not asked	Q7(b), six symbols
Linear probing	Q7(c), size 10	Q7(a), size 5	Hash-table choice Q1(d)
Merge sort	Not asked	Q6(b), eight values	Q4(b), eight values
Loop complexity	Q2(a), product	Q2(a)(i), sum	Q2(a), definitions

"Same operation" is not a claim that the entire printed wording is identical. For example, B Q6(a) and C Q6(a) both delete the first node, but their other insertion tasks differ.

How much of Paper A follows earlier patterns?

An operation-level checklist, beyond a broad topic match

Compare each of A's 16 lettered subparts with the earlier C and B papers together. A **direct method match** means the central requested procedure is explicitly exercised earlier, with different inputs allowed. A **partial match** shares an earlier data structure or some operations, but adds a representation, application, or case not explicitly requested earlier. A **new operation** lacks that specific earlier procedure even though its general data-structure topic is familiar. These are transparent study classifications, not an official measure of copied wording.

A subpart	Classification	Evidence from the earlier papers
Q1(a)	Partial	B Q4(a) has linked-stack push/pop; A requires a bounded array trace including peek.
Q1(b)	Direct method	C Q3(a) uses selection sort; B Q7(c) compares selection and insertion algorithms.
Q1(c)	Direct method	BST construction in C Q4(a), B Q4(b)(i), and B Q5(b).
Q1(d)	Partial	B Q7(a) and C Q7(c) use hashing; A adds ticket-ID lookup and a min-priority scheduling choice.
Q2(a)	Direct method	B Q2(a)(i) asks time/space definitions and loop analysis; C Q2(a) also analyses a loop.
Q2(b)	New operation	Delimiter-type checking with a stack is not explicitly requested in C or B.
Q3(a)	Direct method	Array/list contrast in C Q1(a); insert-after operation in B Q6(a).
Q3(b)	New operation	Earlier BST tasks insert/reconstruct/traverse; they do not delete a BST key.
Q4(a)	Direct method	DFS in C Q1(d) and B Q7(d).
Q4(b)	Direct method	Merge sort with all stages in B Q6(b).
Q5(a)	Direct method	Infix-to-postfix stack conversion in B Q3(b).
Q5(b)	Direct method	Earlier BST insertion and traversal reconstruction supply the procedure; A changes traversal data.
Q6(a)	Direct method	Prim's MST algorithm in B Q5(a), with a different start and graph.
Q6(b)	Partial	B Q4(a) explains doubly linked lists; B Q1(b)/C Q1(c) unlink records; A explicitly adds a general value search and not-found case.
Q7(a)	Direct method	Circular-array queue operations and full/empty checks in C Q2(b); B Q7(b) explains the benefit.
Q7(b)	Direct method	Huffman construction and letter codes in C Q7(b), with changed frequencies.

Result under this rule: 11 of 16 subparts are direct method matches ($\boxed{68.8\%}$), 3 are partial matches ($\boxed{18.8\%}$), and 2 introduce new explicit operations ($\boxed{12.5\%}$). Rounded percentages total 100.1% because of rounding. Combining direct and partial gives $14/16 = \boxed{87.5\%}$ with at least a concrete earlier-method connection. The partial matches are not counted as fully repeated questions.

This is an unweighted count of printed subparts. It does not imply that 87.5% of marks, wording, or difficulty was repeated. In particular, each Q1 lettered subpart is shorter than a 10-mark later subpart. An alternative classification of mixed questions may change these figures; the row-by-row evidence makes that judgement visible.

The shared paper structure and a useful study order

All three papers make Q1 compulsory and ask for any four of the other six questions, giving five attempted questions for 100 marks. Each paper prints seven questions of 20 marks, or 140 available marks including choices. A marks-based overlap must distinguish those 140 available marks from the 100 actually attempted; it cannot assume which options a student selects.

Practise the recurring procedures first: BST insertion and traversals; list pointer changes; expression conversion; circular-queue states; sorting passes; DFS; weighted graph traces; Huffman merges; and hash probing. Then explicitly learn the extensions: BST deletion, delimiter checking, and choosing a structure from required operations. Reproduce states and explain why each update is allowed, rather than memorising final sequences.